Práctica: Comparación de modelos de regresión

Materia: Tópicos de Big Data **Tema:** Introducción al aprendizaje supervisado mediante modelos de regresión **Herramientas:** Python · Jupyter Notebook · Pandas · NumPy · Matplotlib · Scikit-learn

---

1. Objetivo

Construir diferentes modelos de aprendizaje supervisado para realizar **proyecciones del número de nacimientos y defunciones** de un país, utilizando datos históricos.

Durante la práctica se implementarán y compararán visualmente tres modelos:

1. Regresión lineal
2. Regresión polinomial
3. Random Forest Regressor

Al finalizar, podrás observar cómo **diferentes modelos generan predicciones diferentes utilizando exactamente los mismos datos**.

> **Nota:** En esta práctica **no** se realizará todavía la evaluación formal de los modelos. La evaluación mediante métricas se realizará en la siguiente sesión.

---

2. Dataset

Utilizaremos información de **Our World in Data (OWID)** sobre nacimientos y defunciones, que incluye datos históricos y proyecciones hasta el año 2100.

Fuente: *Our World in Data – Births and deaths projected to 2100* (basado en las estimaciones y proyecciones de la ONU).

El dataset contiene información por **país (o región) y año**.

PARTE 1. Importar las bibliotecas

En esta práctica utilizaremos:

- **Pandas** para trabajar con el dataset.
- **NumPy** para trabajar con arreglos y preparar datos.

- **Matplotlib** para visualizar los datos y las predicciones.
- **Scikit-learn** para construir los modelos de Machine Learning.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
!pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in C:\Users\Santiago\Desktop\Escuela\5to
Semestre ISSC\BigData\U1_1\.venv\Lib\site-packages (1.9.1)
Requirement already satisfied: numpy>=1.24.1 in C:\Users\Santiago\Desktop\Escuela\5t
o Semestre ISSC\BigData\U1_1\.venv\Lib\site-packages (from scikit-learn) (2.5.2)
Requirement already satisfied: scipy>=1.10.0 in C:\Users\Santiago\Desktop\Escuela\5t
o Semestre ISSC\BigData\U1_1\.venv\Lib\site-packages (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib>=1.4.0 in C:\Users\Santiago\Desktop\Escuela\5t
o Semestre ISSC\BigData\U1_1\.venv\Lib\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in C:\Users\Santiago\Desktop\Escuela
\5to Semestre ISSC\BigData\U1_1\.venv\Lib\site-packages (from scikit-learn) (2.26.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in C:\Users\Santiago\Desktop\Esc
uela\5to Semestre ISSC\BigData\U1_1\.venv\Lib\site-packages (from scikit-learn) (3.
7.0)
Requirement already satisfied: cloudpickle>=3.0 in C:\Users\Santiago\Desktop\Escuela
\5to Semestre ISSC\BigData\U1_1\.venv\Lib\site-packages (from joblib>=1.4.0->scikit-
learn) (3.1.2)
```

In [3]:
```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.ensemble import RandomForestRegressor
```

PARTE 2. Cargar el dataset

3. Cargar los datos

Leemos el archivo CSV directamente desde la página de OWID.

> **Si no hay internet (o la descarga falla):** descarga el CSV desde la URL de la celda siguiente, guárdalo en la misma carpeta que este notebook con el nombre `births-and-deaths-projected-to-2100.csv` y vuelve a ejecutar la celda. El código lo detecta automáticamente.

In [4]:
```
url = "https://ourworldindata.org/grapher/births-and-deaths-projected-to-2100.csv?v

try:
    df = pd.read_csv(
        url,
        storage_options={
            "User-Agent": "Our World In Data data fetch/1.0"
        }
    )
```

```
except Exception as error:
    print("No fue posible descargar el archivo desde internet:", error)
    print("Se utilizará el archivo local 'births-and-deaths-projected-to-2100.csv'"
    df = pd.read_csv("births-and-deaths-projected-to-2100.csv")

df.head()
```

Out[4]:

	entity	code	year	deaths__sex_all__age_all__variant_estimates	deaths__sex_all__age_a
0	Afghanistan	AFG	1950	290972.0	
1	Afghanistan	AFG	1951	288752.0	
2	Afghanistan	AFG	1952	288059.0	
3	Afghanistan	AFG	1953	287712.0	
4	Afghanistan	AFG	1954	289189.0	

4. Explorar el dataset

Observemos la estructura del `DataFrame`.

In [5]:
```
print("Dimensiones (filas, columnas):", df.shape)
print()
print("Columnas:")
for columna in df.columns:
    print("  -", columna)
```

```
Dimensiones (filas, columnas): (39562, 7)

Columnas:
  - entity
  - code
  - year
  - deaths__sex_all__age_all__variant_estimates
  - deaths__sex_all__age_all__variant_medium__projected
  - births__sex_all__age_all__variant_estimates
  - births__sex_all__age_all__variant_medium__projected
```

In [6]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 39562 entries, 0 to 39561
Data columns (total 7 columns):
 #   Column                                                Non-Null Count  Dtype
---  ------                                                --------------  -----
 0   entity                                                39562 non-null  str
 1   code                                                  38354 non-null  str
 2   year                                                  39562 non-null  int64
 3   deaths__sex_all__age_all__variant_estimates           18944 non-null  float64
 4   deaths__sex_all__age_all__variant_medium__projected  19712 non-null  float64
 5   births__sex_all__age_all__variant_estimates           19166 non-null  float64
 6   births__sex_all__age_all__variant_medium__projected  19943 non-null  float64
dtypes: float64(4), int64(1), str(2)
memory usage: 2.1 MB
```

In [7]:
```
df.head(10)
```

Out[7]:

	entity	code	year	deaths__sex_all__age_all__variant_estimates	deaths__sex_all__age_a
0	Afghanistan	AFG	1950	290972.0	
1	Afghanistan	AFG	1951	288752.0	
2	Afghanistan	AFG	1952	288059.0	
3	Afghanistan	AFG	1953	287712.0	
4	Afghanistan	AFG	1954	289189.0	
5	Afghanistan	AFG	1955	286725.0	
6	Afghanistan	AFG	1956	286654.0	
7	Afghanistan	AFG	1957	286114.0	
8	Afghanistan	AFG	1958	285911.0	
9	Afghanistan	AFG	1959	285500.0	

In [8]:
```
print("Número de entidades:", df["entity"].nunique())
df["entity"].unique()
```

```
Número de entidades: 262
```

Out[8]:
```
<StringArray>
[      'Afghanistan',            'Africa',       'Africa (UN)',
           'Albania',           'Algeria',    'American Samoa',
     'Americas (UN)',           'Andorra',            'Angola',
          'Anguilla',
 ...
           'Vanuatu',           'Vatican',         'Venezuela',
           'Vietnam', 'Wallis and Futuna',    'Western Sahara',
             'World',             'Yemen',            'Zambia',
          'Zimbabwe']
Length: 262, dtype: str
```

Pregunta

¿Cuál es la unidad de observación de cada fila del dataset?

Cada fila es un pais o entidad en un año en especifico.

5. Preparar los nombres de las columnas

Al explorar el dataset notamos dos cosas:

1. Los nombres de las columnas son largos y difíciles de escribir.
2. Los **datos históricos** (`..._variant_estimates`) y las **proyecciones de la ONU** (`..._variant_medium__projected`) vienen en **columnas separadas**. Cuando una fila es histórica, la columna de proyección está vacía (`NaN`), y viceversa.

Vamos a renombrar las columnas y a crear dos columnas unificadas, `births` y `deaths`, que contengan el dato histórico cuando existe y la proyección de OWID cuando no.

In [9]:
```
df = df.rename(columns={
    "entity": "Entity",
    "code": "Code",
    "year": "Year",
    "births__sex_all__age_all__variant_estimates": "births_estimates",
    "births__sex_all__age_all__variant_medium__projected": "births_projected",
    "deaths__sex_all__age_all__variant_estimates": "deaths_estimates",
    "deaths__sex_all__age_all__variant_medium__projected": "deaths_projected",
})


df.head()
```

Out[9]:

	Entity	Code	Year	deaths_estimates	deaths_projected	births_estimates	births_pr
0	Afghanistan	AFG	1950	290972.0	NaN	383985.0	
1	Afghanistan	AFG	1951	288752.0	NaN	391002.0	
2	Afghanistan	AFG	1952	288059.0	NaN	397663.0	
3	Afghanistan	AFG	1953	287712.0	NaN	404666.0	
4	Afghanistan	AFG	1954	289189.0	NaN	410428.0	

In [10]:
```
df_inicial = df.copy()
df_inicial.head(3)
```

Out[10]:

	Entity	Code	Year	deaths_estimates	deaths_projected	births_estimates	births_pro
0	Afghanistan	AFG	1950	290972.0	NaN	383985.0	
1	Afghanistan	AFG	1951	288752.0	NaN	391002.0	
2	Afghanistan	AFG	1952	288059.0	NaN	397663.0	

In [11]:
```
df["births"] = df["births_estimates"].fillna(df["births_projected"])
df["deaths"] = df["deaths_estimates"].fillna(df["deaths_projected"])
df.head()
```

Out[11]:

	Entity	Code	Year	deaths_estimates	deaths_projected	births_estimates	births_pro
0	Afghanistan	AFG	1950	290972.0	NaN	383985.0	
1	Afghanistan	AFG	1951	288752.0	NaN	391002.0	
2	Afghanistan	AFG	1952	288059.0	NaN	397663.0	
3	Afghanistan	AFG	1953	287712.0	NaN	404666.0	
4	Afghanistan	AFG	1954	289189.0	NaN	410428.0	

PARTE 3. Seleccionar un país

6. Seleccionar Japón

Para esta práctica utilizaremos Japón

In [12]:
```
pais = "Japan"

df_japan = df[df["Entity"] == pais]

df_japan.head()
```

Out[12]:

	Entity	Code	Year	deaths_estimates	deaths_projected	births_estimates	births_pro
17063	Japan	JPN	1950	943059.0	NaN	2461534.0	
17064	Japan	JPN	1951	874892.0	NaN	2249284.0	
17065	Japan	JPN	1952	800987.0	NaN	2109219.0	
17066	Japan	JPN	1953	809777.0	NaN	1965540.0	
17067	Japan	JPN	1954	755566.0	NaN	1863833.0	

In [13]:
```
print("Rango de años:", df_japan["Year"].min(), "-", df_japan["Year"].max())
print("Número de filas:", df_japan.shape[0])
```

```
Rango de años: 1950 - 2100
Número de filas: 151
```

7. Revisar las variables disponibles

Identifica las columnas correspondientes a: **año**, **nacimientos** y **defunciones**.

In [14]:
```
df_japan.columns
```

Out[14]:
```
Index(['Entity', 'Code', 'Year', 'deaths_estimates', 'deaths_projected',
       'births_estimates', 'births_projected', 'births', 'deaths'],
      dtype='str')
```

Para esta práctica utilizaremos:

Year
births
deaths

PARTE 4. Explorar visualmente los datos

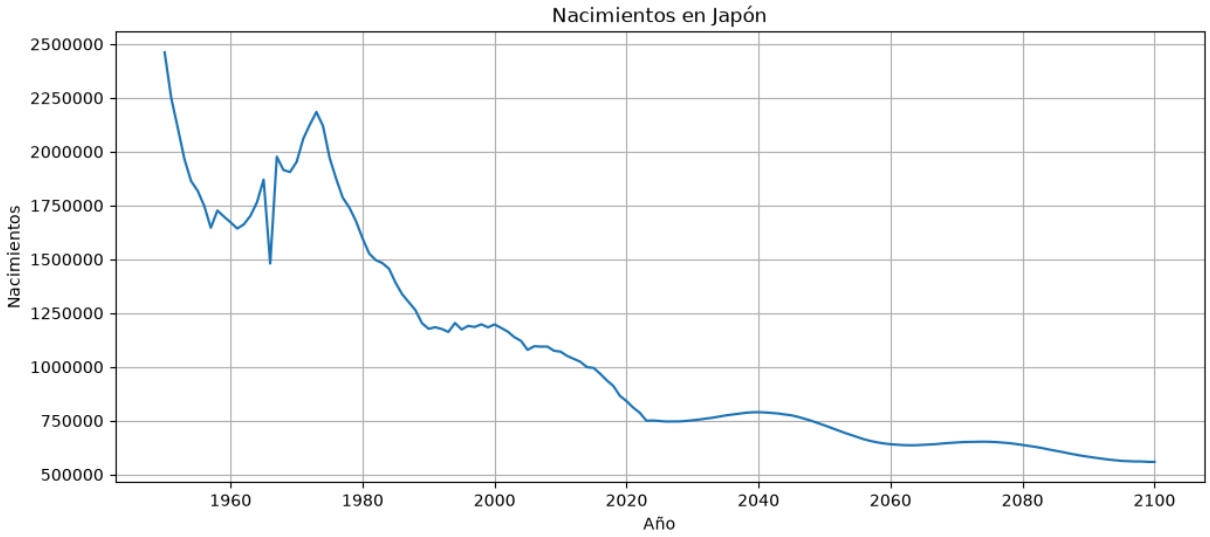
8. Nacimientos a través del tiempo

In [15]:
```
plt.figure(figsize=(12, 5))

plt.plot(
    df_japan["Year"],
    df_japan["births"]
)

plt.title("Nacimientos en Japón")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.grid()
plt.show()
```



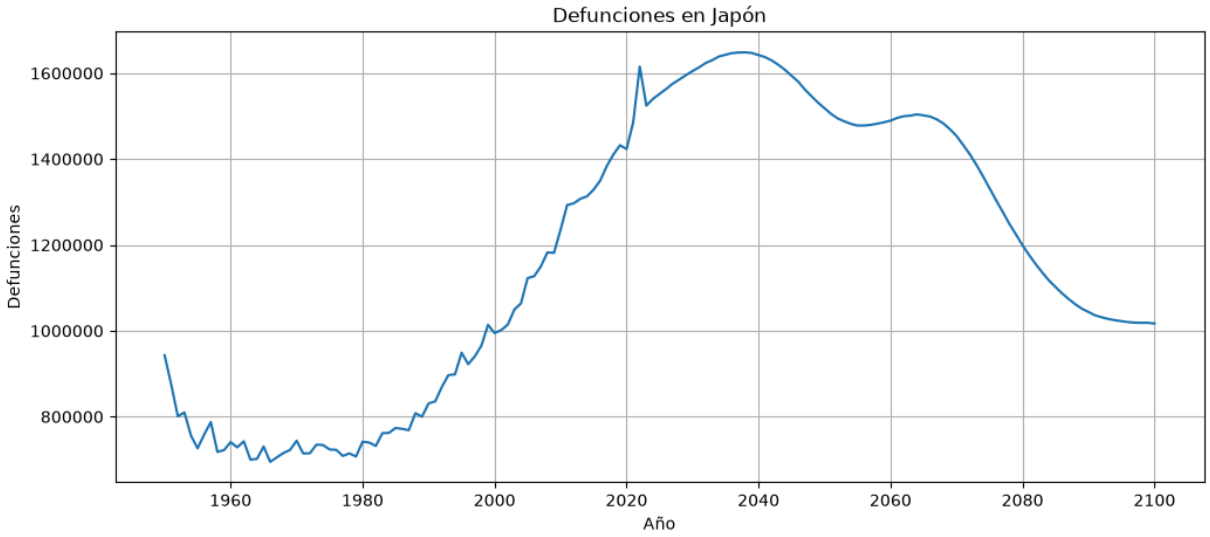
9. Defunciones a través del tiempo

In [16]:
```
plt.figure(figsize=(12, 5))

plt.plot(
    df_japan["Year"],
    df_japan["deaths"]
)

plt.title("Defunciones en Japón")
plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.grid()
plt.show()
```



Preguntas de observación

1. ¿Los nacimientos presentan una tendencia constante? Para nada, no es constante. Sube bastante al principio como por los años 70, llega al pico y despues empieza a caer mucho.
2. ¿Las defunciones presentan una tendencia constante? Tampoco, la verdad se ven muy estables en los primeros años pero luego empiezan a subir cada vez mas rapido.
3. ¿Las dos variables parecen seguir el mismo comportamiento? No, son casi lo contrario, mientras una va para abajo la otra sube.
4. ¿Observas cambios en la tendencia a través del tiempo? Si claro, sobre todo en los nacimientos que se nota un cambio super fuerte cuando dejan de subir y empiezan a caer. En las defunciones es un poco mas gradual.

PARTE 5. Separar datos históricos y proyectados

10. Identificar el punto de separación

El dataset contiene **datos históricos** y **proyecciones**. Antes de separarlos, verifiquemos dónde termina uno y empieza el otro:

In [17]:
```
ultimo_historico = df_japan.loc[df_japan["births_estimates"].notna(), "Year"].max()
primera_proyeccion = df_japan.loc[df_japan["births_projected"].notna(), "Year"].min

print("Último año con dato histórico:", ultimo_historico)
print("Primer año con proyección de OWID:", primera_proyeccion)
```

```
Último año con dato histórico: 2023
Primer año con proyección de OWID: 2024
```

Para esta práctica utilizaremos como punto de separación el **año 2023**. Creamos dos `DataFrames`:

In [18]:
```
df_hist = df_japan[
    df_japan["Year"] <= 2023
]

df_proy = df_japan[
    df_japan["Year"] > 2023
]

print(df_hist.head(3))
print(df_proy.head(3))
```

```
      Entity Code  Year  deaths_estimates  deaths_projected  births_estimates  \
17063  Japan  JPN  1950          943059.0               NaN          2461534.0
17064  Japan  JPN  1951          874892.0               NaN          2249284.0
17065  Japan  JPN  1952          800987.0               NaN          2109219.0

       births_projected     births    deaths
17063               NaN  2461534.0  943059.0
17064               NaN  2249284.0  874892.0
17065               NaN  2109219.0  800987.0
      Entity Code  Year  deaths_estimates  deaths_projected  births_estimates  \
17137  Japan  JPN  2024               NaN         1540205.0               NaN
17138  Japan  JPN  2025               NaN         1551589.0               NaN
17139  Japan  JPN  2026               NaN         1563215.0               NaN

       births_projected    births     deaths
17137          750570.0  750570.0  1540205.0
17138          748163.0  748163.0  1551589.0
17139          745943.0  745943.0  1563215.0
```

In [19]:
```
print("Histórico  ->", df_hist.shape, "| años:", df_hist["Year"].min(), "a", df_his
print("Proyectado ->", df_proy.shape, "| años:", df_proy["Year"].min(), "a", df_pro
```

```
Histórico  -> (74, 9) | años: 1950 a 2023
Proyectado -> (77, 9) | años: 2024 a 2100
```

Comprobemos que no haya valores faltantes en las variables que vamos a utilizar para entrenar:

In [20]:
```
df_hist[["Year", "births", "deaths"]].isna().sum()
```

Out[20]:
```
Year      0
births    0
deaths    0
dtype: int64
```

PARTE 6. Preparar las variables para Machine Learning

11. Definir X e y

Para nuestro primer modelo utilizaremos:

- **X:** año → la **variable predictora**
- **y:** número de nacimientos → la **variable objetivo**

In [21]:
```
X = df_hist[["Year"]]
y_nac = df_hist["births"]
```

In [22]:
```
X.head()
```

Out[22]:

	Year
17063	1950
17064	1951
17065	1952
17066	1953
17067	1954

In [23]:
```
y_nac.head()
```

Out[23]:
```
17063    2461534.0
17064    2249284.0
17065    2109219.0
17066    1965540.0
17067    1863833.0
Name: births, dtype: float64
```

12. Convertir los datos a NumPy

Obtenemos los valores como arreglos de NumPy:

In [24]:
```
X_numpy = X.to_numpy()
y_numpy = y_nac.to_numpy()

print("X_numpy.shape:", X_numpy.shape)
print("y_numpy.shape:", y_numpy.shape)
```

```
X_numpy.shape: (74, 1)
y_numpy.shape: (74,)
```

In [25]:
```
type(X_numpy), type(y_numpy)
```

Out[25]:
```
(numpy.ndarray, numpy.ndarray)
```

Observación

Scikit-learn espera normalmente que las variables predictoras tengan la forma:

`(número de observaciones, número de variables)`
Es decir, que scikit-learn interpreta X como una tabla de datos.

Por eso `X` tiene **dos dimensiones**, mientras que `y` tiene una sola.

En nuestro ejemplo tenemos:

74 observaciones (registros).

1 variable predictora: año (estas son las caracteristicas o atributos usadas para la predicción).

1 variable objetivo: nacimientos (valor que deseamos predecir).

PARTE 7. Modelo 1 – Regresión lineal

13. Crear el modelo

In [26]:
```
mod_lin = LinearRegression()
```

14. Entrenar el modelo

Utilizamos los datos históricos para entrenar el modelo.

In [27]:
```
mod_lin.fit(X, y_nac)
```

Out[27]:
```
LinearRegression
▸ Parameters
▸ Fitted attributes
```

Una regresión lineal aprende una recta `y = a·x + b`. Veamos qué valores aprendió:

In [28]:
```
print("Pendiente (a):", mod_lin.coef_[0])
print("Intercepto (b):", mod_lin.intercept_)
```

```
Pendiente (a): -17650.79986671603
Intercepto (b): 36502903.624420576
```

15. Realizar predicciones

Utilizamos el modelo para estimar los nacimientos correspondientes a los **años históricos**.

In [29]:
```
preds_lin = mod_lin.predict(X)

preds_lin[:10]
```

Out[29]:
```
array([2083843.88432432, 2066193.08445761, 2048542.28459089,
       2030891.48472417, 2013240.68485746, 1995589.88499074,
       1977939.08512402, 1960288.28525731, 1942637.4853906 ,
       1924986.68552388])
```

16. Realizar proyecciones hasta 2100

Creamos los años para los que queremos generar **nuestras propias proyecciones**.

In [30]:
```
anios_futuros = np.arange(2024, 2101).reshape(-1, 1)

print("Forma:", anios_futuros.shape)
anios_futuros[:10]
```

```
Forma: (77, 1)
```

Out[30]:
```
array([[2024],
       [2025],
       [2026],
       [2027],
       [2028],
       [2029],
       [2030],
       [2031],
       [2032],
       [2033]])
```

> **Detalle técnico:** entrenamos el modelo con `X`, que es un `DataFrame` con una columna llamada `Year`. Scikit-learn recuerda ese nombre, así que para predecir también debemos entregarle un `DataFrame` con una columna `Year`. Si le pasáramos un arreglo de NumPy "a secas" mostraría una advertencia (*UserWarning: X does not have valid feature names*). Por eso convertimos los años futuros a `DataFrame`.

In [31]:
```
X_futuro = pd.DataFrame(anios_futuros, columns=["Year"])

X_futuro.head()
```

Out[31]:

	Year
0	2024
1	2025
2	2026
3	2027
4	2028

In [32]:
```
preds_lin_futuras = mod_lin.predict(X_futuro)
```

In [33]:
```
print(preds_lin_futuras[:10])
```

```
[777684.69418734 760033.89432062 742383.0944539  724732.29458719
 707081.49472047 689430.69485375 671779.89498704 654129.09512033
 636478.2952536  618827.49538689]
```

PARTE 8. Visualizar la regresión lineal

17. Comparar datos históricos y predicciones

In [34]:
```
plt.figure(figsize=(12, 6))

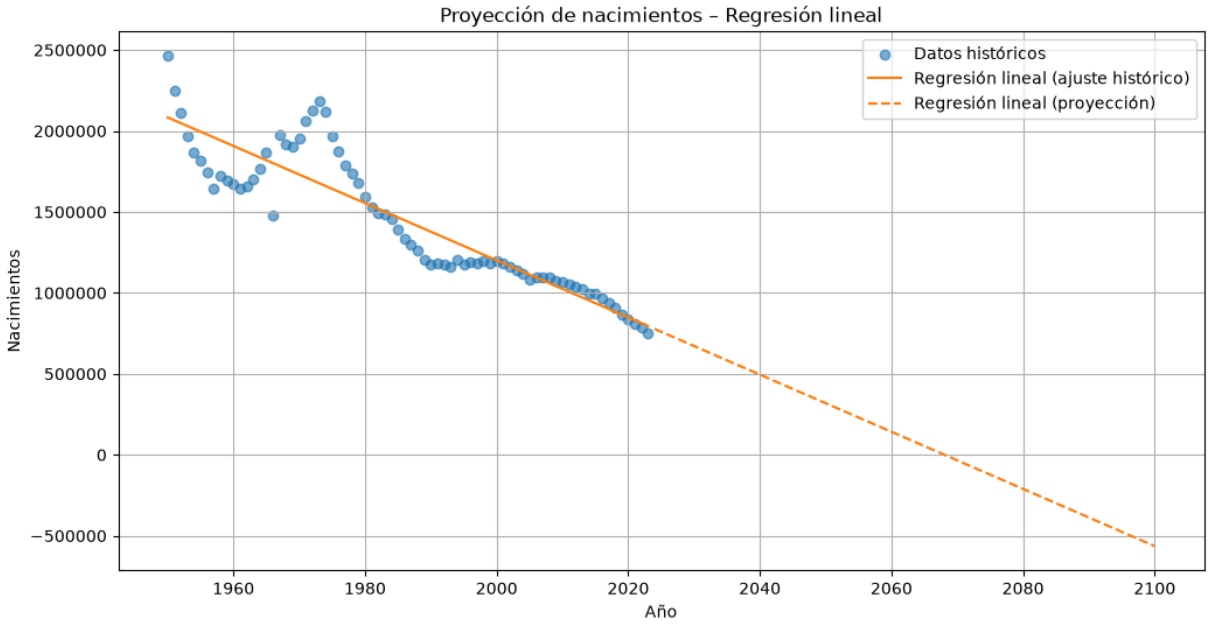
plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

# Ajuste del modelo sobre los años históricos
plt.plot(
    df_hist["Year"],
    preds_lin,
    color="tab:orange",
    label="Regresión lineal (ajuste histórico)"
)

# Proyección hacia el futuro
plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    linestyle="--",
    label="Regresión lineal (proyección)"
)

plt.title("Proyección de nacimientos – Regresión lineal")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Analiza cómo continúa la tendencia histórica cuando utilizamos una **función lineal**. ¿Qué pasa con los valores hacia el año 2100? Pues la regresion lineal solo dibuja una linea recta hacia abajo ignorando que la caida no es tan recta. Hacia el 2100 predice que los nacimientos van a seguir bajando hasta llegar a cero o a negativos, lo cual no tiene sentido en la vida real.

PARTE 9. Modelo 2 – Regresión polinomial

18. Crear variables polinomiales

La regresión lineal solamente puede representar una relación **lineal** entre las variables. Ahora construiremos variables polinomiales para permitir **curvas**. Utilizaremos un polinomio de **grado 2**:

$$y = a \cdot x^2 + b \cdot x + c$$

In [35]:
```
poly = PolynomialFeatures(
    degree=2,
    include_bias=False
)
```

Transformamos `X`:

In [36]:
```
X_poly = poly.fit_transform(X)

X_poly[:5]
```

Out[36]:
```
array([[1.950000e+03, 3.802500e+06],
       [1.951000e+03, 3.806401e+06],
       [1.952000e+03, 3.810304e+06],
       [1.953000e+03, 3.814209e+06],
       [1.954000e+03, 3.818116e+06]])
```

Observa que ahora cada año tiene **dos columnas**: el año (`x`) y el año al cuadrado (`x²`).

19. Crear y entrenar el modelo polinomial

Seguimos utilizando `LinearRegression` ; lo único que cambió son las **variables de entrada**.

In [37]:
```
mod_poli = LinearRegression()

mod_poli.fit(
    X_poly,
    y_nac
)
```

Out[37]:

▾ LinearRegression ⓘ ?

▸ Parameters

▸ Fitted attributes

20. Generar predicciones históricas

In [38]:
```
preds_poli = mod_poli.predict(
    X_poly
)
```

21. Generar proyecciones hasta 2100

Transformamos los años futuros utilizando **el mismo objeto** `poly` con el que transformamos los datos de entrenamiento.

In [39]:
```
anios_futuros_poly = poly.transform(
    X_futuro
)

preds_poli_futuras = mod_poli.predict(
    anios_futuros_poly
)
```

PARTE 10. Visualizar la regresión polinomial

22. Graficar las predicciones

In [40]:
```
plt.figure(figsize=(12, 6))

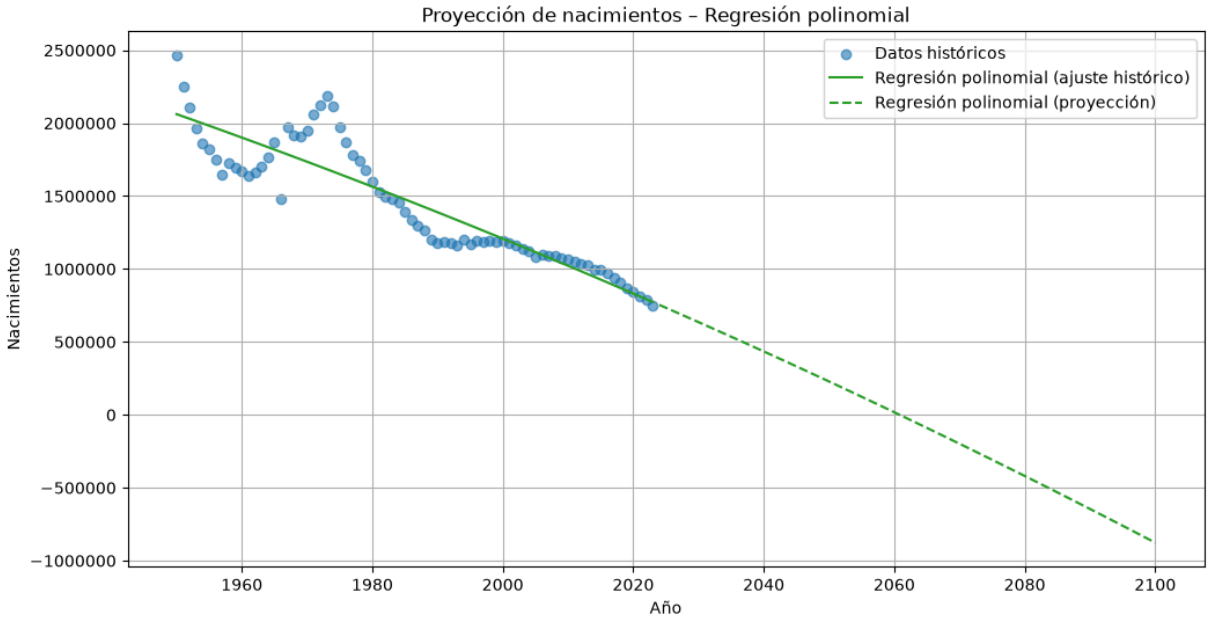
plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

plt.plot(
    df_hist["Year"],
    preds_poli,
    color="tab:green",
    label="Regresión polinomial (ajuste histórico)"
)

plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    linestyle="--",
    label="Regresión polinomial (proyección)"
)

plt.title("Proyección de nacimientos – Regresión polinomial")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Compara el comportamiento de esta gráfica con el modelo de regresión lineal. ¿Se ajusta mejor a los datos históricos? Si se ve mucho mejor porque la curvita se va adaptando a las variaciones de los años pasados. A diferencia de la recta, copia un poco mejor la subida y la caida.

¿Qué ocurre con la proyección hacia el año 2100? ¿Los valores que predice tienen sentido para un número de nacimientos? Aqui ya se va todo a la basura jaja. La proyeccion se va super rapido para abajo a numeros negativos desde mucho antes del 2100. Es obvio que no tiene sentido tener -1 millón de nacimientos, la regresion polinomial exagera mucho la caida y se vuelve inutil.

PARTE 11. Modelo 3 – Random Forest

23. Crear el modelo

Un **Random Forest** está formado por muchos árboles de decisión. Utilizaremos `RandomForestRegressor`.

In [41]:
```
mod_rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
```

24. Entrenar el modelo

In [42]:
```
mod_rf.fit(
    X,
    y_nac
)
```

Out[42]:

RandomForestRegressor

▶ Parameters

▶ Fitted attributes

25. Generar predicciones históricas

In [43]:
```
preds_rf = mod_rf.predict(X)
```

26. Generar proyecciones futuras

In [44]:
```
preds_rf_futuras = mod_rf.predict(
    X_futuro
)
```

PARTE 12. Visualizar Random Forest

27. Graficar las predicciones

In [45]:
```
plt.figure(figsize=(12, 6))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

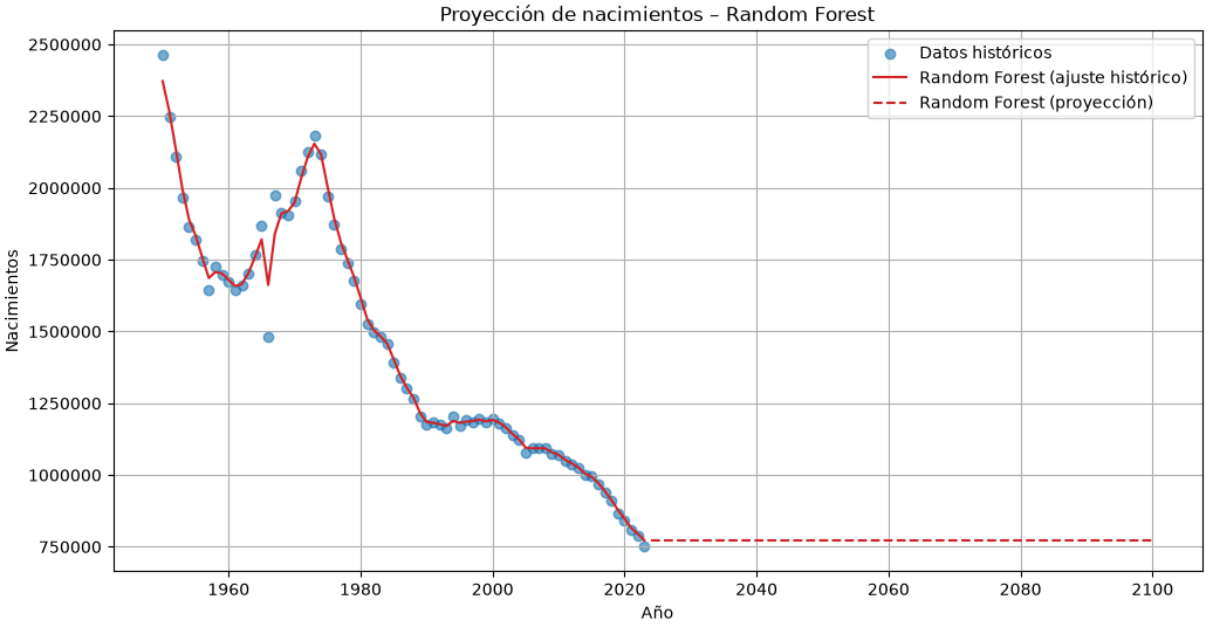
plt.plot(
    df_hist["Year"],
    preds_rf,
    color="tab:red",
    label="Random Forest (ajuste histórico)"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    linestyle="--",
```

```
    label="Random Forest (proyección)"
)

plt.title("Proyección de nacimientos – Random Forest")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Fíjate en dos cosas: (1) qué tan bien sigue el modelo los datos históricos y (2) qué sucede con la línea después de 2023. ¿Continúa la tendencia? ¿Por qué crees que sucede esto? Investiga.

1. El modelo sigue los datos historicos como si fuera un calcado. Pasa por cada puntito sin fallar (creo que a esto le dicen overfitting o algo asi).
2. Despues del 2023 la linea se queda completamente plana. No sigue ni para arriba ni para abajo. Me puse a buscar y al parecer los random forest no pueden predecir mas alla del rango que ya vieron, asi que solo repiten el ultimo valor que aprendieron.

PARTE 13. Comparar los tres modelos

28. Graficar los tres modelos

Construimos una gráfica que permita comparar directamente los resultados.

> Si la regresión polinomial genera valores muy extremos y no permite ver bien las demás curvas, puedes limitar el eje vertical agregando, por ejemplo, `plt.ylim(0, 3_500_000)` antes de `plt.show()`.

In [46]:
```
plt.figure(figsize=(14, 7))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.5
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    label="Regresión lineal"
)

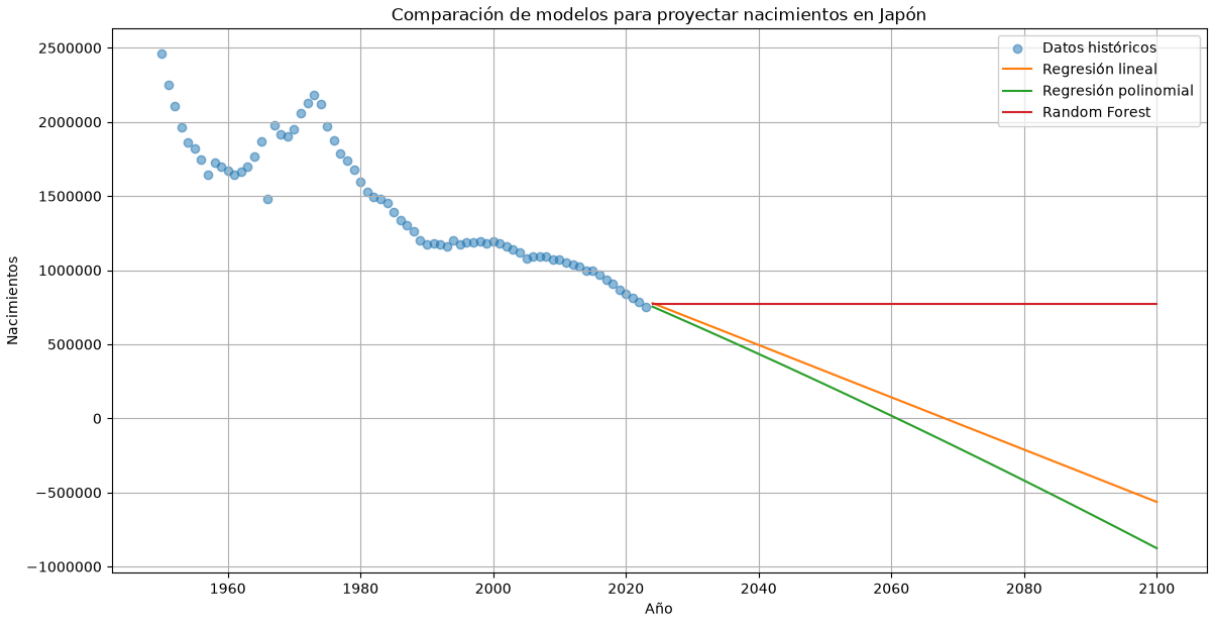
plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Comparación de modelos para proyectar nacimientos en Japón"
)

plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



PARTE 14. Repetir el proceso para defunciones

Hasta este momento construimos modelos para predecir **nacimientos**. Ahora repetiremos el mismo proceso para **defunciones**.

29. Preparar la variable objetivo

La variable predictora sigue siendo `X` (el año). Solo cambia la variable objetivo:

In [47]:
```
y_def = df_hist["deaths"]
```

30. Regresión lineal para defunciones

In [48]:
```
mod_lin_deaths = LinearRegression()

mod_lin_deaths.fit(
    X,
    y_def
)

preds_lin_deaths = mod_lin_deaths.predict(
    X_futuro
)
```

31. Regresión polinomial para defunciones

Utilizamos nuevamente las variables construidas con `PolynomialFeatures` (`X_poly` y `anios_futuros_poly`).

In [49]:
```
mod_poli_deaths = LinearRegression()

mod_poli_deaths.fit(
    X_poly,
    y_def
)

preds_poli_deaths = mod_poli_deaths.predict(
    anios_futuros_poly
)
```

32. Random Forest para defunciones

In [50]:
```
mod_rf_deaths = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

mod_rf_deaths.fit(
    X,
    y_def
)

preds_rf_deaths = mod_rf_deaths.predict(
    X_futuro
)
```

PARTE 15. Comparar las predicciones de defunciones

33. Visualizar los tres modelos

In [51]:
```
plt.figure(figsize=(14, 7))

plt.scatter(
    df_hist["Year"],
    df_hist["deaths"],
    label="Datos históricos",
    alpha=0.5
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_deaths,
    color="tab:orange",
    label="Regresión lineal"
```

```
)

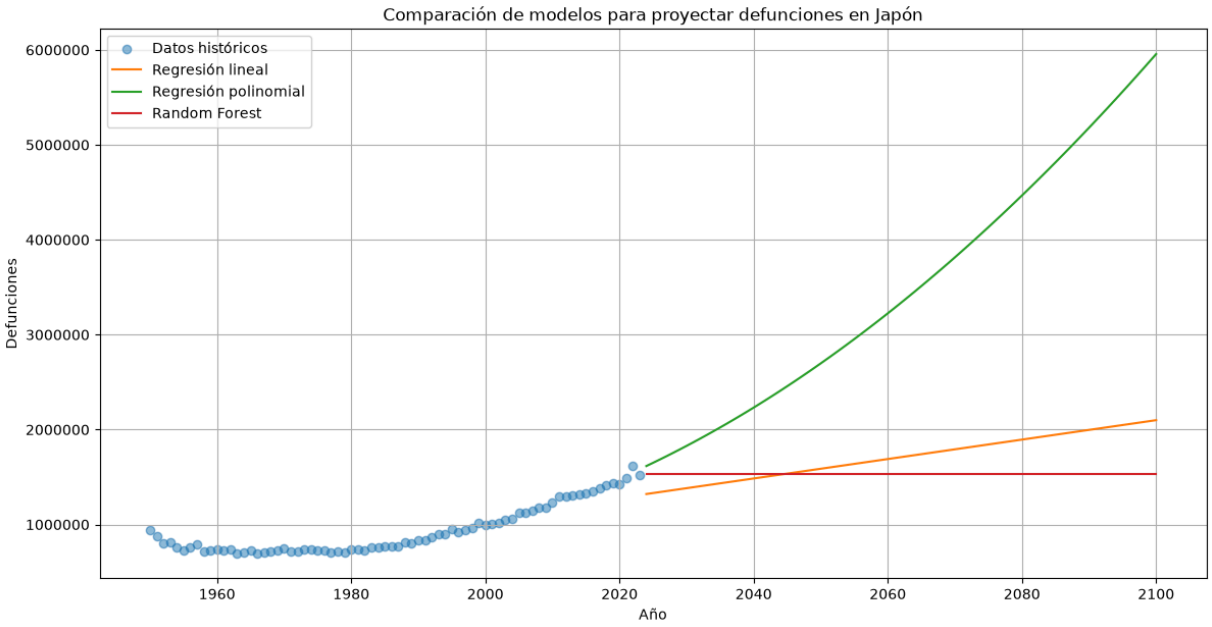
plt.plot(
    anios_futuros.flatten(),
    preds_poli_deaths,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_deaths,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Comparación de modelos para proyectar defunciones en Japón"
)

plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



PARTE 16. Comparar nuestros modelos con las proyecciones de OWID

Finalmente, agregamos a las gráficas las **proyecciones proporcionadas por OWID** (las de la ONU, escenario medio). Estas proyecciones no se obtienen ajustando una recta a la serie

histórica: se construyen con modelos demográficos que consideran fecundidad, mortalidad y estructura por edad de la población.

34. Nacimientos: ML vs OWID

In [52]:
```
plt.figure(figsize=(14, 7))

plt.plot(
    df_proy["Year"],
    df_proy["births"],
    color="black",
    linewidth=2.5,
    label="Proyección OWID"
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    label="Regresión lineal"
)

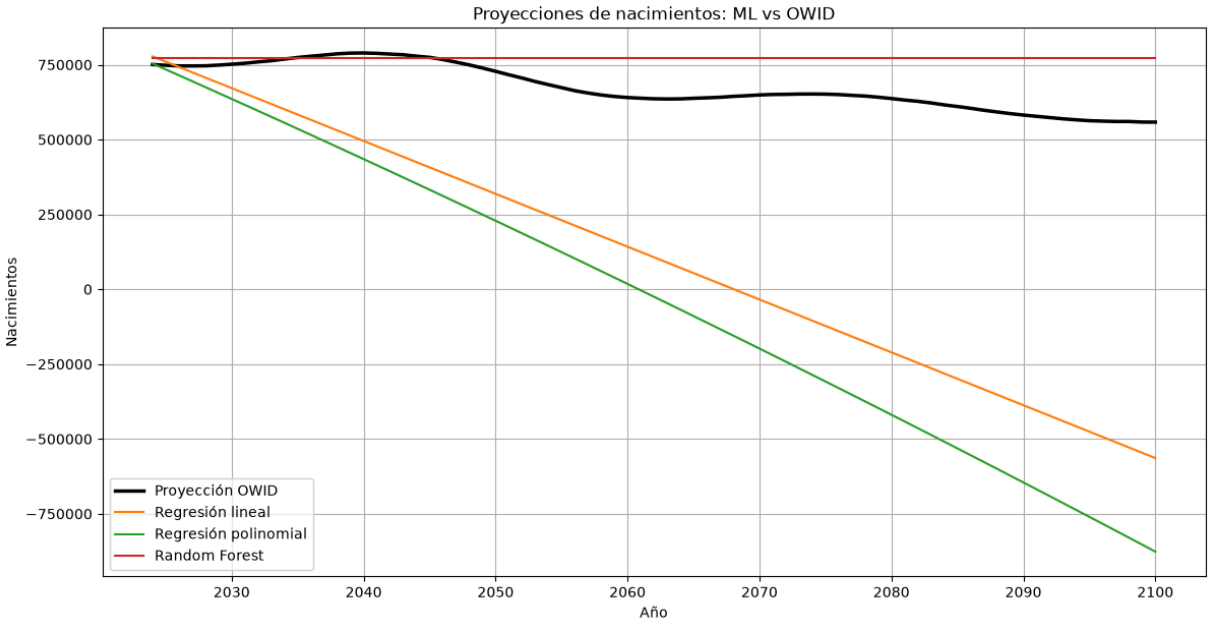
plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Proyecciones de nacimientos: ML vs OWID"
)

plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



35. Defunciones: ML vs OWID

Repetimos el ejercicio para defunciones.

In [53]:
```
plt.figure(figsize=(14, 7))

plt.plot(
    df_proy["Year"],
    df_proy["deaths"],
    color="black",
    linewidth=2.5,
    label="Proyección OWID"
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_deaths,
    color="tab:orange",
    label="Regresión lineal"
)

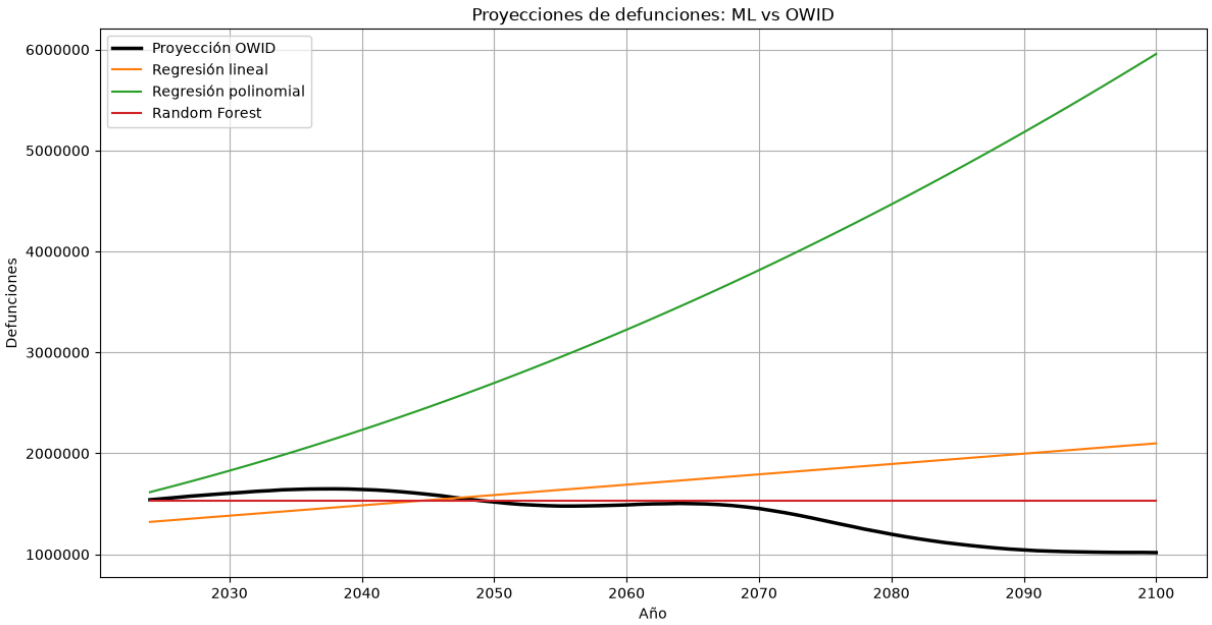
plt.plot(
    anios_futuros.flatten(),
    preds_poli_deaths,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_deaths,
    color="tab:red",
    label="Random Forest"
)
```

```
plt.title(
    "Proyecciones de defunciones: ML vs OWID"
)

plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



36. Resumen numérico (opcional)

A veces es útil ver los números y no solo la gráfica. Esta tabla muestra lo que predice cada modelo para algunos años seleccionados:

In [54]:
```
anios_tabla = [2030, 2050, 2075, 2100]
indice = anios_futuros.flatten()


def construir_tabla(owid, lineal, polinomial, rf):
    tabla = pd.DataFrame(
        {
            "OWID": owid.set_index("Year").reindex(indice).to_numpy().ravel(),
            "Regresión lineal": lineal,
            "Regresión polinomial": polinomial,
            "Random Forest": rf,
        },
        index=indice
    )
    tabla.index.name = "Año"
    return tabla.loc[anios_tabla]
```

```
print("NACIMIENTOS")
display(
    construir_tabla(
        df_proy[["Year", "births"]],
        preds_lin_futuras,
        preds_poli_futuras,
        preds_rf_futuras
    ).style.format("{:,.0f}")
)

print("DEFUNCIONES")
display(
    construir_tabla(
        df_proy[["Year", "deaths"]],
        preds_lin_deaths,
        preds_poli_deaths,
        preds_rf_deaths
    ).style.format("{:,.0f}")
)
```

NACIMIENTOS

Año	OWID	Regresión lineal	Regresión polinomial	Random Forest
2030	751,702	671,780	635,660	772,259
2050	727,944	318,764	228,816	772,259
2075	651,691	-122,506	-308,036	772,259
2100	558,209	-563,776	-876,329	772,259

DEFUNCIONES

Año	OWID	Regresión lineal	Regresión polinomial	Random Forest
2030	1,604,957	1,382,640	1,828,332	1,528,838
2050	1,518,396	1,587,377	2,697,262	1,528,838
2075	1,332,059	1,843,298	4,132,590	1,528,838
2100	1,016,915	2,099,220	5,955,882	1,528,838

PARTE 17. Reflexión

Responde las siguientes preguntas en esta misma celda (haz doble clic para editarla).

1.

¿Qué diferencia observaste entre la regresión lineal, la regresión polinomial y Random Forest?

Respuesta: La lineal solo te avienta una recta sin importarle las curvas de los datos reales. La polinomial si trata de hacer las curvas pero al final de los datos se dispara hacia los negativos sin control. Y el random forest copio todo perfecto en el pasado pero hacia el futuro se rinde y se queda plano.

2.

¿Por qué los tres modelos generan predicciones diferentes si utilizaron los mismos datos históricos?

Respuesta: Porque cada uno usa logicas distintas. La lineal busca una recta promedio, la polinomial busca una funcion con curvas que pase por los puntos y el random forest usa arboles de decision que solo pueden aprender de rangos que ya conocen.

3.

¿Qué modelo produjo una tendencia más cercana visualmente a las proyecciones de OWID?

Respuesta: Ninguno es perfecto pero quiza la polinomial al inicio del pronostico. El problema es que despues cae en picada muy feo, asi que a la larga la regresion lineal como que se ve un poco menos peor, aunque las de OWID tienen una curva mas natural y realista.

4.

¿El modelo que visualmente parece acercarse más a las proyecciones de OWID necesariamente es el mejor modelo? Explica.

Respuesta: No, a simple vista pueden engañar. Aparte un modelo como el random forest que se ve "perfecto" en los datos pasados resulta ser inutil para el futuro. Siempre hay que checar si los resultados tienen sentido logico y evaluar con metricas de error, no solo que la grafica se vea bonita.

5.

¿Qué comportamiento observaste en las predicciones de Random Forest para los años posteriores a 2023?

Respuesta: Literalmente se hizo una linea horizontal. Esto pasa porque el algoritmo no sabe predecir cosas nuevas fuera del rango que se le dio para entrenar.

6.

¿Por qué debemos tener cuidado al utilizar un modelo entrenado con datos históricos para realizar predicciones hasta el año 2100?

Respuesta: Porque son 80 años hacia el futuro. Los modelos matematicos solo estiran la tendencia que traen sin pensar en cosas reales como economia, enfermedades o politicas (cosas que un buen pronostico poblacional si toma en cuenta).

PARTE 18. Conceptos clave

Al finalizar la práctica, debes poder identificar:

Concepto	Significado
Variable predictora	Variable utilizada por el modelo para generar una predicción
Variable objetivo	Variable que queremos predecir
Entrenamiento	Proceso mediante el cual el modelo aprende a partir de los datos
Predicción	Resultado generado por el modelo
Regresión lineal	Modelo que representa una relación lineal
Regresión polinomial	Extensión que permite representar relaciones curvas
Random Forest	Modelo basado en múltiples árboles de decisión
Extrapolación	Generación de predicciones fuera del rango de datos utilizados para entrenar el modelo

> **Importante:** En esta práctica solamente hemos **construido y utilizado** los modelos. En la siguiente práctica aprenderemos a **evaluar su desempeño mediante métricas**, para determinar qué tan bien están realizando sus predicciones.

Ejercicio 1: Extracción del país de China

PARTE 3. Seleccionar un país

6. Seleccionar China

Para esta práctica utilizaremos China

In [55]:
```
pais = "China"

df_china = df[df["Entity"] == pais]

df_china.head()
```

Out[55]:

	Entity	Code	Year	deaths_estimates	deaths_projected	births_estimates	births_proj
7097	China	CHN	1950	12564023.0	NaN	22293376.0	
7098	China	CHN	1951	12411701.0	NaN	22144814.0	
7099	China	CHN	1952	12459848.0	NaN	25496318.0	
7100	China	CHN	1953	12309525.0	NaN	24139416.0	
7101	China	CHN	1954	12145578.0	NaN	25438544.0	

In [56]:
```
print("Rango de años:", df_china["Year"].min(), "-", df_china["Year"].max())
print("Número de filas:", df_china.shape[0])
```

```
Rango de años: 1950 - 2100
Número de filas: 151
```

7. Revisar las variables disponibles

Identifica las columnas correspondientes a: **año**, **nacimientos** y **defunciones**.

In [57]:
```
df_china.columns
```

Out[57]:
```
Index(['Entity', 'Code', 'Year', 'deaths_estimates', 'deaths_projected',
       'births_estimates', 'births_projected', 'births', 'deaths'],
      dtype='str')
```

Para esta práctica utilizaremos:

```
Year
births
deaths
```

PARTE 4. Explorar visualmente los datos

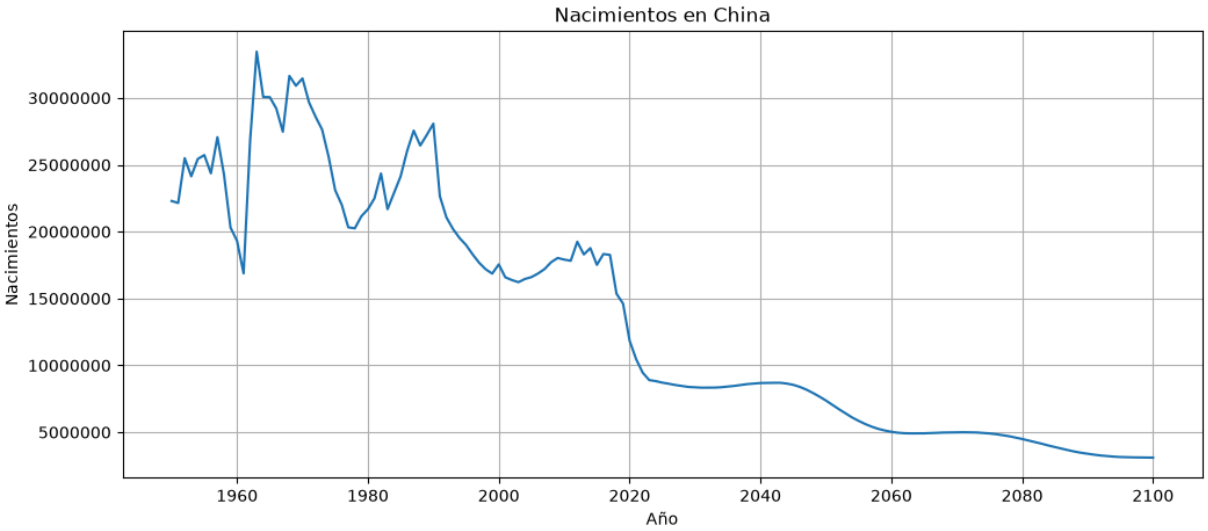
8. Nacimientos a través del tiempo

In [58]:
```
plt.figure(figsize=(12, 5))

plt.plot(
    df_china["Year"],
    df_china["births"]
)

plt.title("Nacimientos en China")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.grid()
plt.show()
```



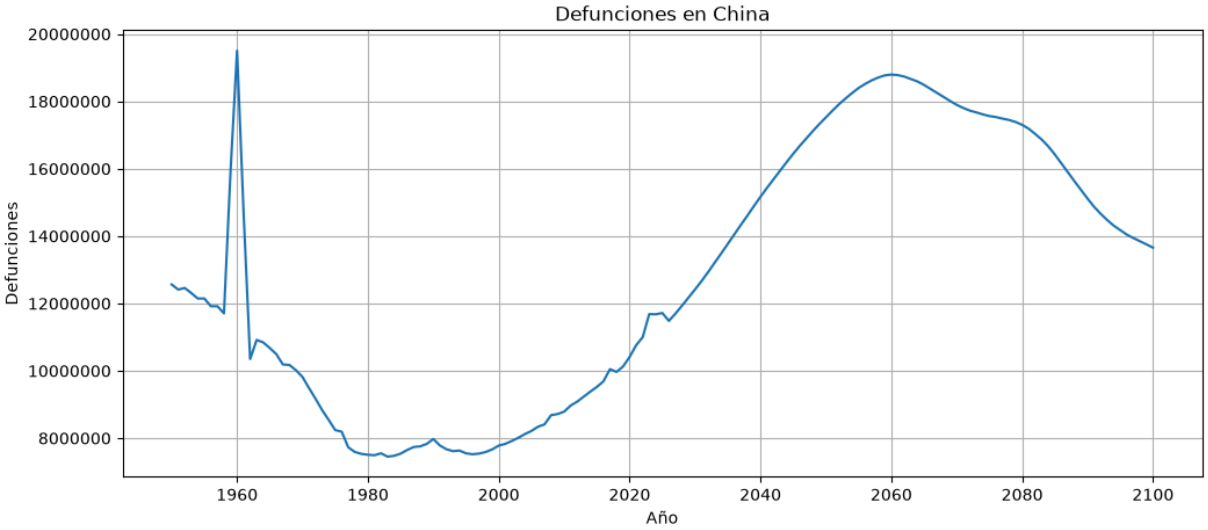
9. Defunciones a través del tiempo

In [59]:
```
plt.figure(figsize=(12, 5))

plt.plot(
    df_china["Year"],
    df_china["deaths"]
)

plt.title("Defunciones en China")
plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.grid()
plt.show()
```



Preguntas de observación

1. ¿Los nacimientos presentan una tendencia constante? Para nada, no es constante. Sube bastante al principio como por los años 70, llega al pico y despues empieza a caer mucho.
2. ¿Las defunciones presentan una tendencia constante? Tampoco, la verdad se ven muy estables en los primeros años pero luego empiezan a subir cada vez mas rapido.
3. ¿Las dos variables parecen seguir el mismo comportamiento? No, son casi lo contrario, mientras una va para abajo la otra sube.
4. ¿Observas cambios en la tendencia a través del tiempo? Si claro, sobre todo en los nacimientos que se nota un cambio super fuerte cuando dejan de subir y empiezan a caer. En las defunciones es un poco mas gradual.

PARTE 5. Separar datos históricos y proyectados

10. Identificar el punto de separación

El dataset contiene **datos históricos** y **proyecciones**. Antes de separarlos, verifiquemos dónde termina uno y empieza el otro:

In [60]:
```
ultimo_historico = df_china.loc[df_china["births_estimates"].notna(), "Year"].max()
primera_proyeccion = df_china.loc[df_china["births_projected"].notna(), "Year"].min

print("Último año con dato histórico:", ultimo_historico)
print("Primer año con proyección de OWID:", primera_proyeccion)
```

```
Último año con dato histórico: 2023
Primer año con proyección de OWID: 2024
```

Para esta práctica utilizaremos como punto de separación el **año 2023**. Creamos dos `DataFrames`:

In [61]:
```
df_hist = df_china[
    df_china["Year"] <= 2023
]

df_proy = df_china[
    df_china["Year"] > 2023
]

print(df_hist.head(3))
print(df_proy.head(3))
```

```
     Entity Code  Year  deaths_estimates  deaths_projected  births_estimates  \
7097  China  CHN  1950        12564023.0               NaN        22293376.0
7098  China  CHN  1951        12411701.0               NaN        22144814.0
7099  China  CHN  1952        12459848.0               NaN        25496318.0

      births_projected      births      deaths
7097               NaN  22293376.0  12564023.0
7098               NaN  22144814.0  12411701.0
7099               NaN  25496318.0  12459848.0
     Entity Code  Year  deaths_estimates  deaths_projected  births_estimates  \
7171  China  CHN  2024               NaN        11678348.0               NaN
7172  China  CHN  2025               NaN        11716328.0               NaN
7173  China  CHN  2026               NaN        11482042.0               NaN

      births_projected     births      deaths
7171         8822056.0  8822056.0  11678348.0
7172         8709352.0  8709352.0  11716328.0
7173         8625232.0  8625232.0  11482042.0
```

In [62]:
```
print("Histórico  ->", df_hist.shape, "| años:", df_hist["Year"].min(), "a", df_his
print("Proyectado ->", df_proy.shape, "| años:", df_proy["Year"].min(), "a", df_pro
```

```
Histórico  -> (74, 9) | años: 1950 a 2023
Proyectado -> (77, 9) | años: 2024 a 2100
```

Comprobemos que no haya valores faltantes en las variables que vamos a utilizar para entrenar:

In [63]:
```
df_hist[["Year", "births", "deaths"]].isna().sum()
```

Out[63]:
```
Year      0
births    0
deaths    0
dtype: int64
```

PARTE 6. Preparar las variables para Machine Learning

11. Definir X e y

Para nuestro primer modelo utilizaremos:

- **X:** año → la **variable predictora**
- **y:** número de nacimientos → la **variable objetivo**

In [64]:
```
X = df_hist[["Year"]]
y_nac = df_hist["births"]
```

In [65]:
```
X.head()
```

Out[65]:

	Year
7097	1950
7098	1951
7099	1952
7100	1953
7101	1954

In [66]:
```
y_nac.head()
```

Out[66]:
```
7097    22293376.0
7098    22144814.0
7099    25496318.0
7100    24139416.0
7101    25438544.0
Name: births, dtype: float64
```

12. Convertir los datos a NumPy

Obtenemos los valores como arreglos de NumPy:

In [67]:
```
X_numpy = X.to_numpy()
y_numpy = y_nac.to_numpy()

print("X_numpy.shape:", X_numpy.shape)
print("y_numpy.shape:", y_numpy.shape)
```

```
X_numpy.shape: (74, 1)
y_numpy.shape: (74,)
```

In [68]:
```
type(X_numpy), type(y_numpy)
```

Out[68]:
```
(numpy.ndarray, numpy.ndarray)
```

Observación

Scikit-learn espera normalmente que las variables predictoras tengan la forma:

`(número de observaciones, número de variables)`
Es decir, que scikit-learn interpreta X como una tabla de datos.

Por eso `X` tiene **dos dimensiones**, mientras que `y` tiene una sola.

En nuestro ejemplo tenemos:

74 observaciones (registros).

1 variable predictora: año (estas son las caracteristicas o atributos usadas para la predicción).

1 variable objetivo: nacimientos (valor que deseamos predecir).

PARTE 7. Modelo 1 – Regresión lineal

13. Crear el modelo

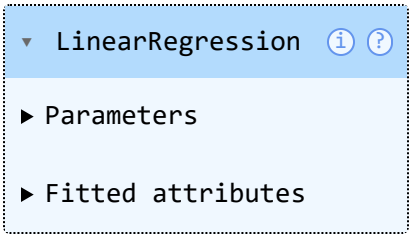
In [69]:
```
mod_lin = LinearRegression()
```

14. Entrenar el modelo

Utilizamos los datos históricos para entrenar el modelo.

In [70]:
```
mod_lin.fit(X, y_nac)
```

Out[70]:



Una regresión lineal aprende una recta `y = a·x + b`. Veamos qué valores aprendió:

In [71]:
```
print("Pendiente (a):", mod_lin.coef_[0])
print("Intercepto (b):", mod_lin.intercept_)
```

```
Pendiente (a): -187208.31004813028
Intercepto (b): 393489519.9376378
```

15. Realizar predicciones

Utilizamos el modelo para estimar los nacimientos correspondientes a los **años históricos**.

In [72]:
```
preds_lin = mod_lin.predict(X)

preds_lin[:10]
```

Out[72]:
```
array([28433315.34378374, 28246107.03373563, 28058898.72368747,
       27871690.41363937, 27684482.10359126, 27497273.7935431 ,
       27310065.483495  , 27122857.17344683, 26935648.86339873,
       26748440.55335057])
```

16. Realizar proyecciones hasta 2100

Creamos los años para los que queremos generar **nuestras propias proyecciones**.

In [73]:
```
anios_futuros = np.arange(2024, 2101).reshape(-1, 1)

print("Forma:", anios_futuros.shape)
anios_futuros[:10]
```

```
Forma: (77, 1)
```

Out[73]:
```
array([[2024],
       [2025],
       [2026],
       [2027],
       [2028],
       [2029],
       [2030],
       [2031],
       [2032],
       [2033]])
```

> **Detalle técnico:** entrenamos el modelo con `X`, que es un `DataFrame` con una columna llamada `Year`. Scikit-learn recuerda ese nombre, así que para predecir también debemos entregarle un `DataFrame` con una columna `Year`. Si le pasáramos un arreglo de NumPy "a secas" mostraría una advertencia (*UserWarning: X does not have valid feature names*). Por eso convertimos los años futuros a `DataFrame`.

In [74]:
```
X_futuro = pd.DataFrame(anios_futuros, columns=["Year"])

X_futuro.head()
```

Out[74]:

	Year
0	2024
1	2025
2	2026
3	2027
4	2028

In [75]:
```
preds_lin_futuras = mod_lin.predict(X_futuro)
```

In [76]:
```
print(preds_lin_futuras[:10])
```

```
[14579900.40022212 14392692.09017396 14205483.78012586 14018275.47007769
 13831067.16002959 13643858.84998149 13456650.53993332 13269442.22988522
 13082233.91983706 12895025.60978895]
```

PARTE 8. Visualizar la regresión lineal

17. Comparar datos históricos y predicciones

In [77]:
```
plt.figure(figsize=(12, 6))

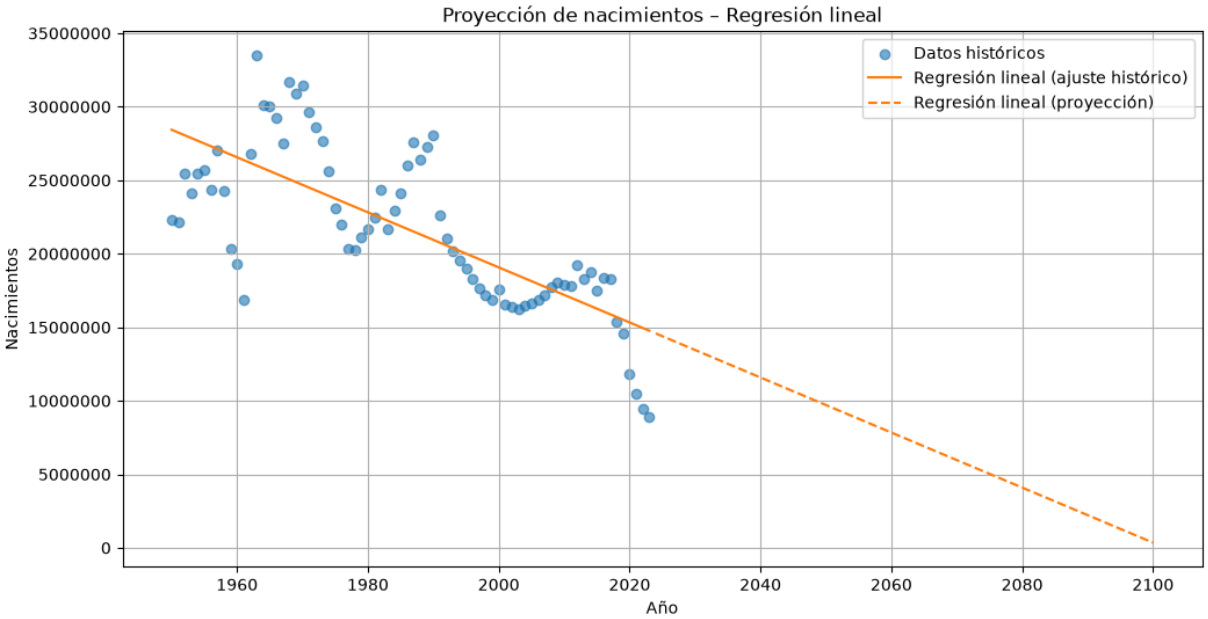
plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

# Ajuste del modelo sobre los años históricos
plt.plot(
    df_hist["Year"],
    preds_lin,
    color="tab:orange",
    label="Regresión lineal (ajuste histórico)"
)

# Proyección hacia el futuro
plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    linestyle="--",
    label="Regresión lineal (proyección)"
)

plt.title("Proyección de nacimientos – Regresión lineal")
plt.xlabel("Año")
plt.ylabel("Nacimientos")
```

```
plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Analiza cómo continúa la tendencia histórica cuando utilizamos una **función lineal**. ¿Qué pasa con los valores hacia el año 2100? Pues la regresion lineal solo dibuja una linea recta hacia abajo ignorando que la caida no es tan recta. Hacia el 2100 predice que los nacimientos van a seguir bajando hasta llegar a cero o a negativos, lo cual no tiene sentido en la vida real.

PARTE 9. Modelo 2 – Regresión polinomial

18. Crear variables polinomiales

La regresión lineal solamente puede representar una relación **lineal** entre las variables. Ahora construiremos variables polinomiales para permitir **curvas**. Utilizaremos un polinomio de **grado 2**:

$$y = a \cdot x^2 + b \cdot x + c$$

In [78]:
```
poly = PolynomialFeatures(
    degree=2,
    include_bias=False
)
```

Transformamos `X`:

In [79]:
```
X_poly = poly.fit_transform(X)

X_poly[:5]
```

Out[79]:
```
array([[1.950000e+03, 3.802500e+06],
       [1.951000e+03, 3.806401e+06],
       [1.952000e+03, 3.810304e+06],
       [1.953000e+03, 3.814209e+06],
       [1.954000e+03, 3.818116e+06]])
```

Observa que ahora cada año tiene **dos columnas**: el año (`x`) y el año al cuadrado (`x²`).

19. Crear y entrenar el modelo polinomial

Seguimos utilizando `LinearRegression`; lo único que cambió son las **variables de entrada**.

In [80]:
```
mod_poli = LinearRegression()

mod_poli.fit(
    X_poly,
    y_nac
)
```

Out[80]:

▾ LinearRegression ⓘ ?

▸ Parameters

▸ Fitted attributes

20. Generar predicciones históricas

In [81]:
```
preds_poli = mod_poli.predict(
    X_poly
)
```

21. Generar proyecciones hasta 2100

Transformamos los años futuros utilizando **el mismo objeto** `poly` con el que transformamos los datos de entrenamiento.

In [82]:
```
anios_futuros_poly = poly.transform(
    X_futuro
)
```

```
preds_poli_futuras = mod_poli.predict(
    anios_futuros_poly
)
```

PARTE 10. Visualizar la regresión polinomial

22. Graficar las predicciones

In [83]:
```
plt.figure(figsize=(12, 6))

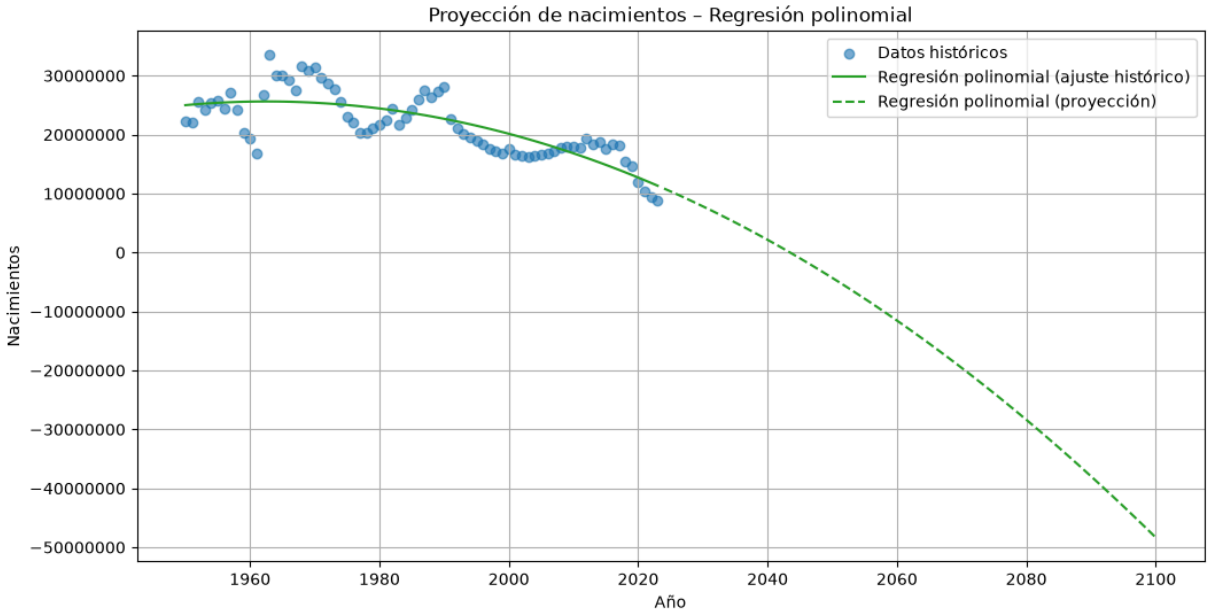
plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

plt.plot(
    df_hist["Year"],
    preds_poli,
    color="tab:green",
    label="Regresión polinomial (ajuste histórico)"
)

plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    linestyle="--",
    label="Regresión polinomial (proyección)"
)

plt.title("Proyección de nacimientos – Regresión polinomial")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Compara el comportamiento de esta gráfica con el modelo de regresión lineal. ¿Se ajusta mejor a los datos históricos? Si se ve mucho mejor porque la curvita se va adaptando a las variaciones de los años pasados. A diferencia de la recta, copia un poco mejor la subida y la caida.

¿Qué ocurre con la proyección hacia el año 2100? ¿Los valores que predice tienen sentido para un número de nacimientos? Aqui ya se va todo a la basura jaja. La proyeccion se va super rapido para abajo a numeros negativos desde mucho antes del 2100. Es obvio que no tiene sentido tener -1 millón de nacimientos, la regresion polinomial exagera mucho la caida y se vuelve inutil.

PARTE 11. Modelo 3 – Random Forest

23. Crear el modelo

Un **Random Forest** está formado por muchos árboles de decisión. Utilizaremos `RandomForestRegressor`.

In [84]:
```
mod_rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
```

24. Entrenar el modelo

In [85]:
```
mod_rf.fit(
    X,
    y_nac
)
```

Out[85]:

RandomForestRegressor

▶ Parameters

▶ Fitted attributes

25. Generar predicciones históricas

In [86]:
```
preds_rf = mod_rf.predict(X)
```

26. Generar proyecciones futuras

In [87]:
```
preds_rf_futuras = mod_rf.predict(
    X_futuro
)
```

PARTE 12. Visualizar Random Forest

27. Graficar las predicciones

In [88]:
```
plt.figure(figsize=(12, 6))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

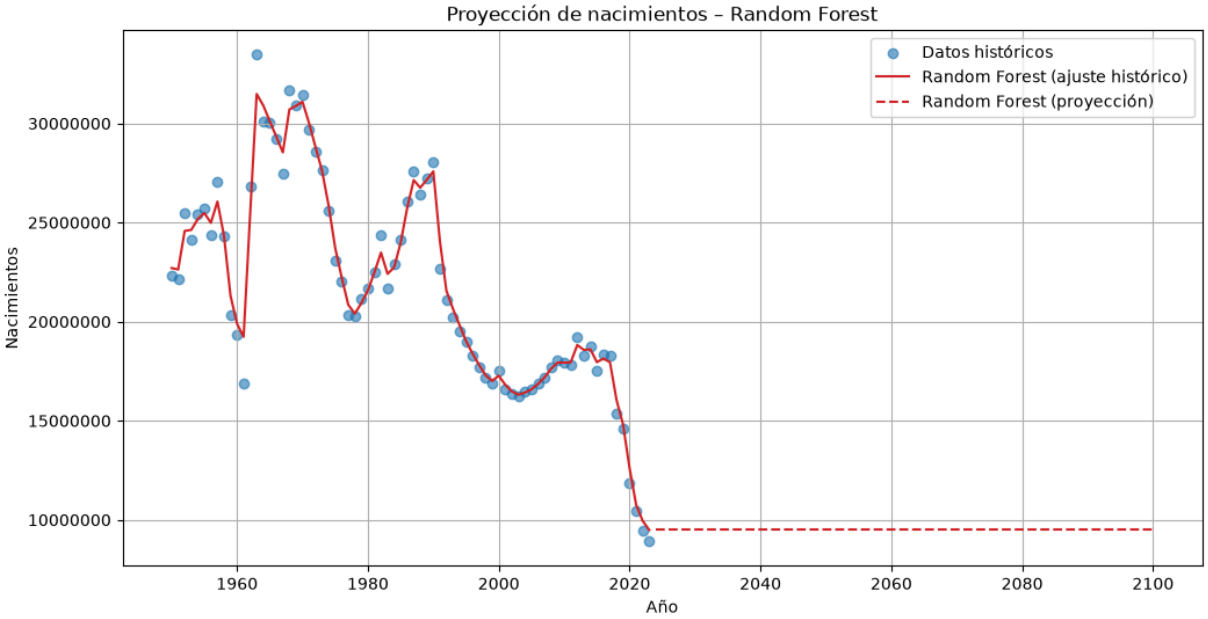
plt.plot(
    df_hist["Year"],
    preds_rf,
    color="tab:red",
    label="Random Forest (ajuste histórico)"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    linestyle="--",
```

```
    label="Random Forest (proyección)"
)

plt.title("Proyección de nacimientos – Random Forest")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Fíjate en dos cosas: (1) qué tan bien sigue el modelo los datos históricos y (2) qué sucede con la línea después de 2023. ¿Continúa la tendencia? ¿Por qué crees que sucede esto? Investiga.

1. El modelo sigue los datos historicos como si fuera un calcado. Pasa por cada puntito sin fallar (creo que a esto le dicen overfitting o algo asi).
2. Despues del 2023 la linea se queda completamente plana. No sigue ni para arriba ni para abajo. Me puse a buscar y al parecer los random forest no pueden predecir mas alla del rango que ya vieron, asi que solo repiten el ultimo valor que aprendieron.

PARTE 13. Comparar los tres modelos

28. Graficar los tres modelos

Construimos una gráfica que permita comparar directamente los resultados.

> Si la regresión polinomial genera valores muy extremos y no permite ver bien las demás curvas, puedes limitar el eje vertical agregando, por ejemplo, `plt.ylim(0, 3_500_000)` antes de `plt.show()`.

In [89]:
```
plt.figure(figsize=(14, 7))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.5
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    label="Regresión lineal"
)

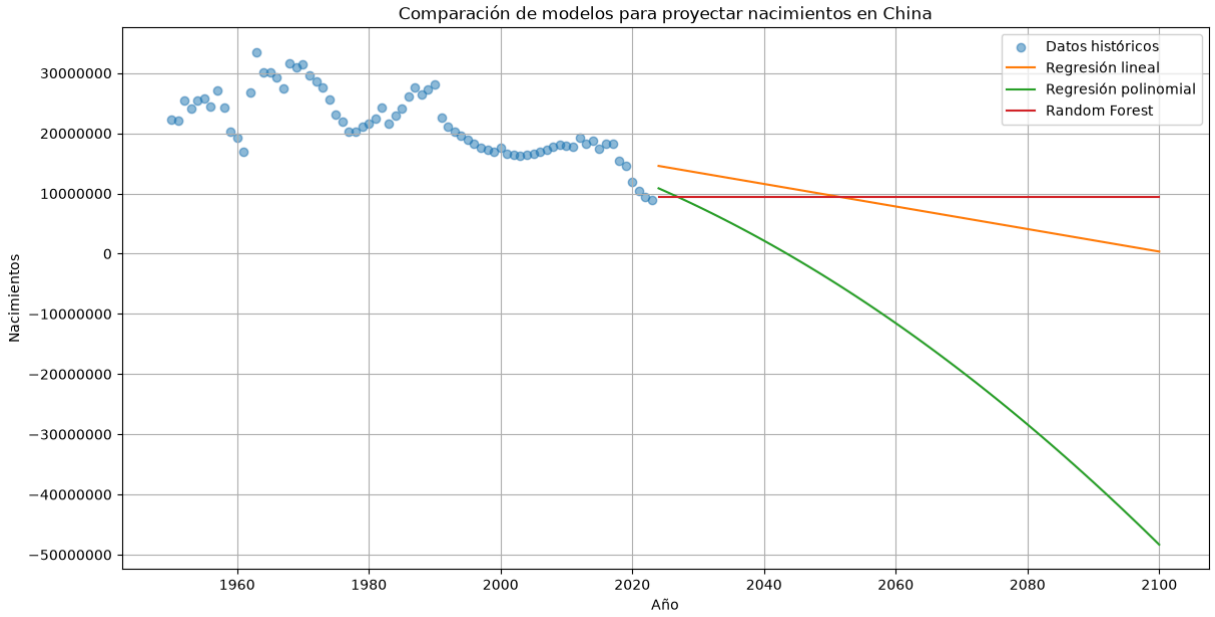
plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Comparación de modelos para proyectar nacimientos en China"
)

plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



PARTE 14. Repetir el proceso para defunciones

Hasta este momento construimos modelos para predecir **nacimientos**. Ahora repetiremos el mismo proceso para **defunciones**.

29. Preparar la variable objetivo

La variable predictora sigue siendo `X` (el año). Solo cambia la variable objetivo:

In [90]:
```
y_def = df_hist["deaths"]
```

30. Regresión lineal para defunciones

In [91]:
```
mod_lin_deaths = LinearRegression()

mod_lin_deaths.fit(
    X,
    y_def
)

preds_lin_deaths = mod_lin_deaths.predict(
    X_futuro
)
```

31. Regresión polinomial para defunciones

Utilizamos nuevamente las variables construidas con `PolynomialFeatures` (`X_poly` y `anios_futuros_poly`).

In [92]:
```
mod_poli_deaths = LinearRegression()

mod_poli_deaths.fit(
    X_poly,
    y_def
)

preds_poli_deaths = mod_poli_deaths.predict(
    anios_futuros_poly
)
```

32. Random Forest para defunciones

In [93]:
```
mod_rf_deaths = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

mod_rf_deaths.fit(
    X,
    y_def
)

preds_rf_deaths = mod_rf_deaths.predict(
    X_futuro
)
```

PARTE 15. Comparar las predicciones de defunciones

33. Visualizar los tres modelos

In [94]:
```
plt.figure(figsize=(14, 7))

plt.scatter(
    df_hist["Year"],
    df_hist["deaths"],
    label="Datos históricos",
    alpha=0.5
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_deaths,
    color="tab:orange",
    label="Regresión lineal"
```

```
)

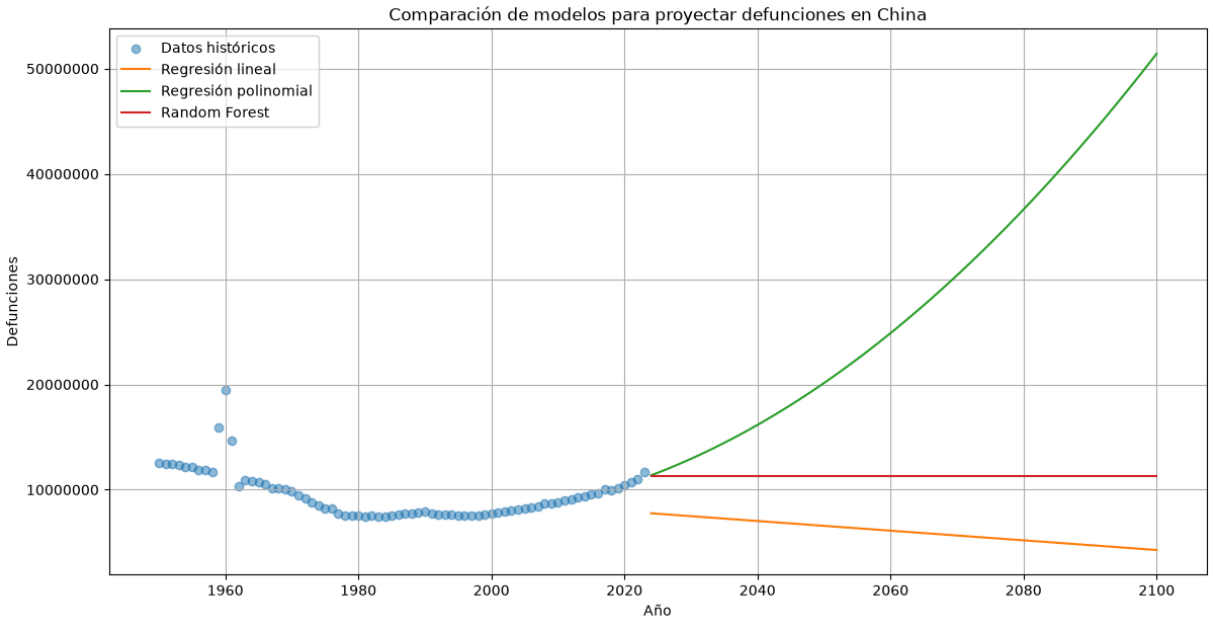
plt.plot(
    anios_futuros.flatten(),
    preds_poli_deaths,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_deaths,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Comparación de modelos para proyectar defunciones en China"
)

plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



PARTE 16. Comparar nuestros modelos con las proyecciones de OWID

Finalmente, agregamos a las gráficas las **proyecciones proporcionadas por OWID** (las de la ONU, escenario medio). Estas proyecciones no se obtienen ajustando una recta a la serie

histórica: se construyen con modelos demográficos que consideran fecundidad, mortalidad y estructura por edad de la población.

34. Nacimientos: ML vs OWID

In [95]:
```
plt.figure(figsize=(14, 7))

plt.plot(
    df_proy["Year"],
    df_proy["births"],
    color="black",
    linewidth=2.5,
    label="Proyección OWID"
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    label="Regresión lineal"
)

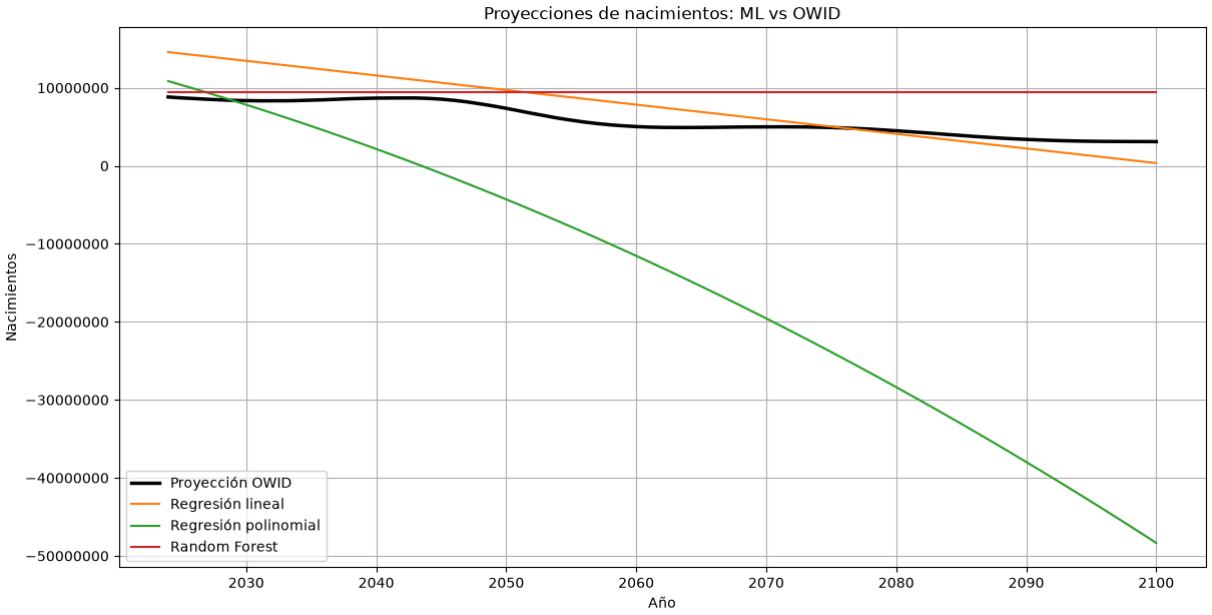
plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Proyecciones de nacimientos: ML vs OWID"
)

plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



35. Defunciones: ML vs OWID

Repetimos el ejercicio para defunciones.

In [96]:
```
plt.figure(figsize=(14, 7))

plt.plot(
    df_proy["Year"],
    df_proy["deaths"],
    color="black",
    linewidth=2.5,
    label="Proyección OWID"
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_deaths,
    color="tab:orange",
    label="Regresión lineal"
)

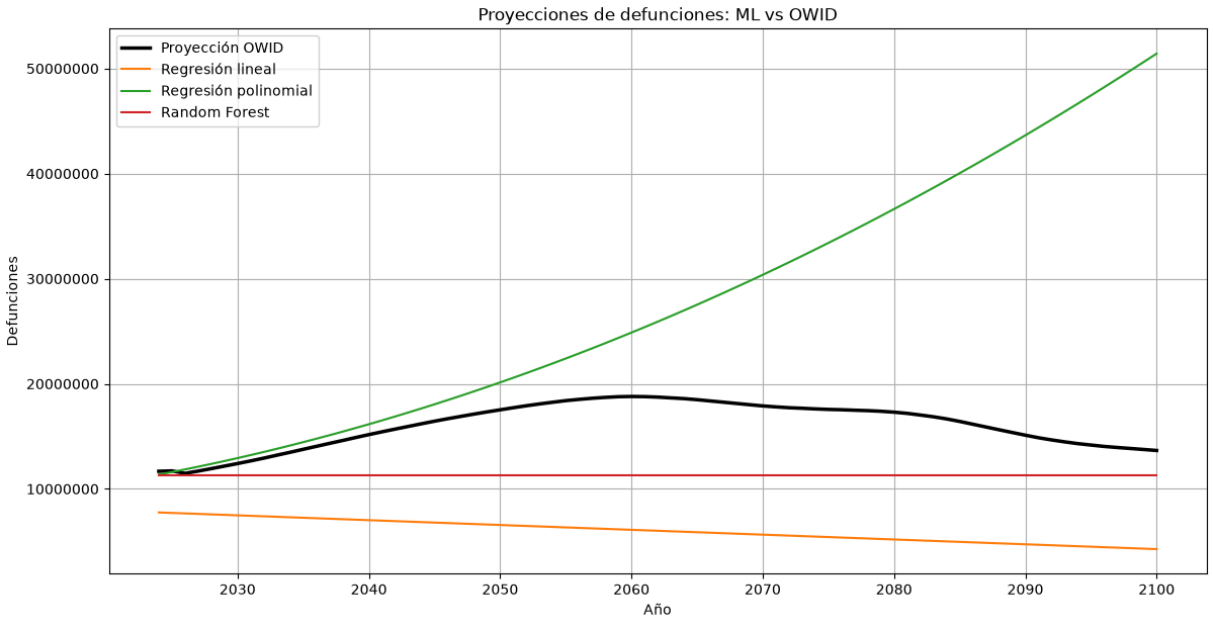
plt.plot(
    anios_futuros.flatten(),
    preds_poli_deaths,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_deaths,
    color="tab:red",
    label="Random Forest"
)
```

```
plt.title(
    "Proyecciones de defunciones: ML vs OWID"
)

plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



36. Resumen numérico (opcional)

A veces es útil ver los números y no solo la gráfica. Esta tabla muestra lo que predice cada modelo para algunos años seleccionados:

In [97]:
```
anios_tabla = [2030, 2050, 2075, 2100]
indice = anios_futuros.flatten()


def construir_tabla(owid, lineal, polinomial, rf):
    tabla = pd.DataFrame(
        {
            "OWID": owid.set_index("Year").reindex(indice).to_numpy().ravel(),
            "Regresión lineal": lineal,
            "Regresión polinomial": polinomial,
            "Random Forest": rf,
        },
        index=indice
    )
    tabla.index.name = "Año"
    return tabla.loc[anios_tabla]
```

```
print("NACIMIENTOS")
display(
    construir_tabla(
        df_proy[["Year", "births"]],
        preds_lin_futuras,
        preds_poli_futuras,
        preds_rf_futuras
    ).style.format("{:,.0f}")
)

print("DEFUNCIONES")
display(
    construir_tabla(
        df_proy[["Year", "deaths"]],
        preds_lin_deaths,
        preds_poli_deaths,
        preds_rf_deaths
    ).style.format("{:,.0f}")
)
```

NACIMIENTOS

Año	OWID	Regresión lineal	Regresión polinomial	Random Forest
2030	8,359,513	13,456,651	7,830,199	9,489,420
2050	7,371,348	9,712,484	-4,298,790	9,489,420
2075	4,909,046	5,032,277	-23,867,936	9,489,420
2100	3,099,844	352,069	-48,334,758	9,489,420

DEFUNCIONES

Año	OWID	Regresión lineal	Regresión polinomial	Random Forest
2030	12,415,078	7,485,699	12,933,834	11,339,159
2050	17,526,300	6,567,577	20,134,799	11,339,159
2075	17,564,112	5,419,925	33,404,217	11,339,159
2100	13,655,445	4,272,272	51,416,093	11,339,159

==========

Ejercicio 2: Extracción del país de México (a partir del año 2000)

==========

PARTE 3. Seleccionar un país

6. Seleccionar México

Para esta práctica utilizaremos México

In [98]:
```
pais = "Mexico"

df_mexico = df[df["Entity"] == pais]

# Filtrar a partir del año 2000
df_mexico = df_mexico[df_mexico["Year"] >= 2000]

df_mexico.head()
```

Out[98]:

	Entity	Code	Year	deaths_estimates	deaths_projected	births_estimates	births_pr
22851	Mexico	MEX	2000	540567.0	NaN	2385977.0	
22852	Mexico	MEX	2001	543803.0	NaN	2380261.0	
22853	Mexico	MEX	2002	553391.0	NaN	2374140.0	
22854	Mexico	MEX	2003	563140.0	NaN	2351177.0	
22855	Mexico	MEX	2004	575925.0	NaN	2335226.0	

In [99]:
```
print("Rango de años:", df_mexico["Year"].min(), "-", df_mexico["Year"].max())
print("Número de filas:", df_mexico.shape[0])
```

```
Rango de años: 2000 - 2100
Número de filas: 101
```

7. Revisar las variables disponibles

Identifica las columnas correspondientes a: **año**, **nacimientos** y **defunciones**.

In [100…
```
df_mexico.columns
```

Out[100…
```
Index(['Entity', 'Code', 'Year', 'deaths_estimates', 'deaths_projected',
       'births_estimates', 'births_projected', 'births', 'deaths'],
      dtype='str')
```

Para esta práctica utilizaremos:

Year
births
deaths

PARTE 4. Explorar visualmente los datos

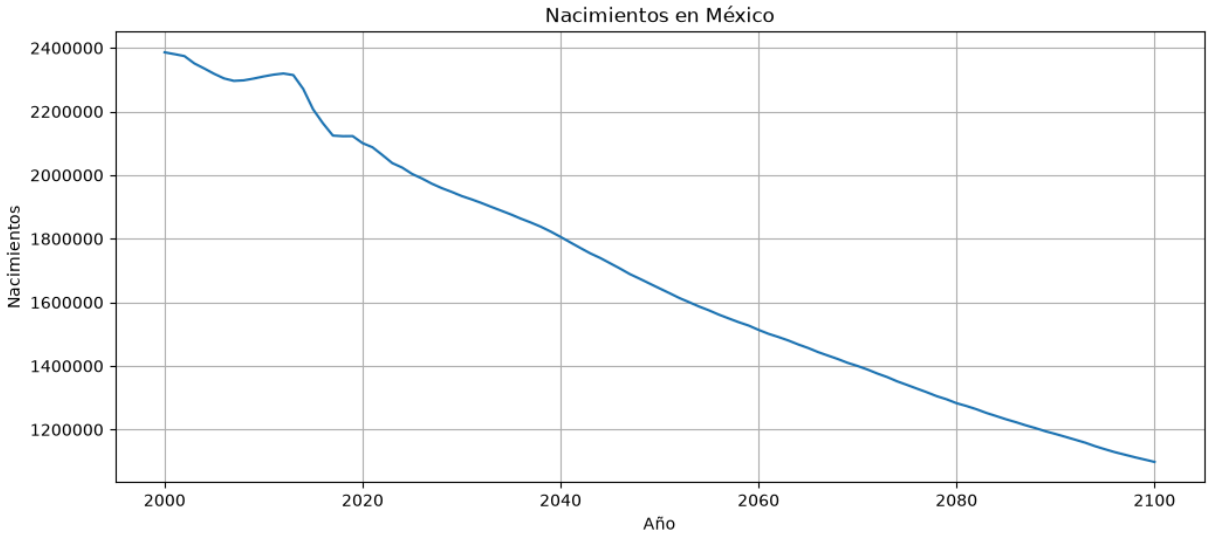
8. Nacimientos a través del tiempo

In [101…
```
plt.figure(figsize=(12, 5))

plt.plot(
    df_mexico["Year"],
    df_mexico["births"]
)

plt.title("Nacimientos en México")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.grid()
plt.show()
```



9. Defunciones a través del tiempo

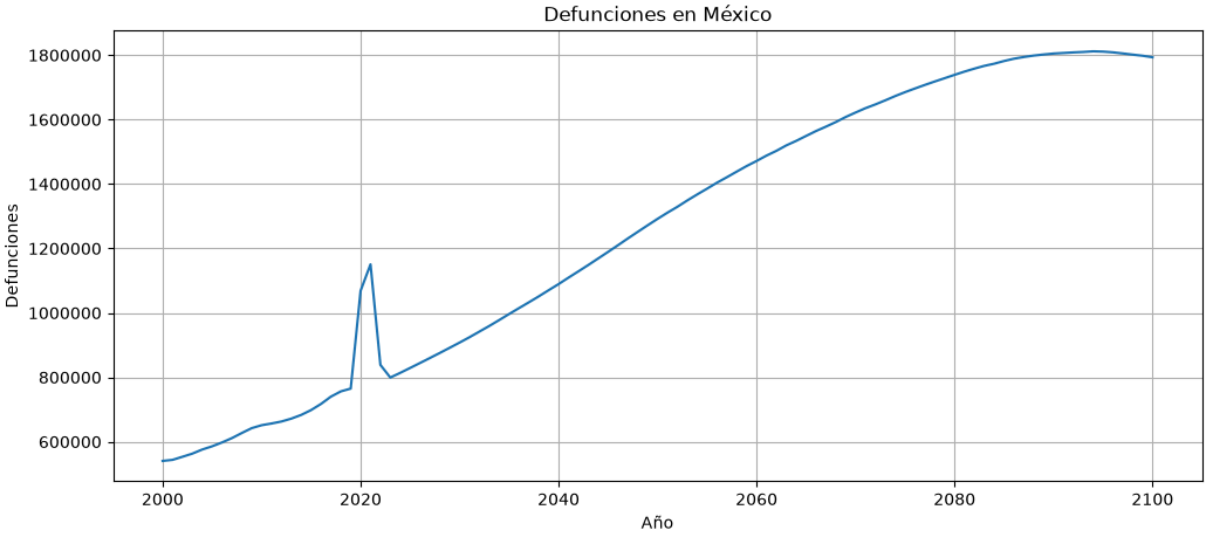
In [102…

```
plt.figure(figsize=(12, 5))

plt.plot(
    df_mexico["Year"],
    df_mexico["deaths"]
)

plt.title("Defunciones en México")
plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.grid()
plt.show()
```



Preguntas de observación

1. ¿Los nacimientos presentan una tendencia constante? Para nada, no es constante. Sube bastante al principio como por los años 70, llega al pico y despues empieza a caer mucho.
2. ¿Las defunciones presentan una tendencia constante? Tampoco, la verdad se ven muy estables en los primeros años pero luego empiezan a subir cada vez mas rapido.
3. ¿Las dos variables parecen seguir el mismo comportamiento? No, son casi lo contrario, mientras una va para abajo la otra sube.
4. ¿Observas cambios en la tendencia a través del tiempo? Si claro, sobre todo en los nacimientos que se nota un cambio super fuerte cuando dejan de subir y empiezan a caer. En las defunciones es un poco mas gradual.

PARTE 5. Separar datos históricos y proyectados

10. Identificar el punto de separación

El dataset contiene **datos históricos** y **proyecciones**. Antes de separarlos, verifiquemos dónde termina uno y empieza el otro:

In [103…
```
ultimo_historico = df_mexico.loc[df_mexico["births_estimates"].notna(), "Year"].max
primera_proyeccion = df_mexico.loc[df_mexico["births_projected"].notna(), "Year"].m

print("Último año con dato histórico:", ultimo_historico)
print("Primer año con proyección de OWID:", primera_proyeccion)
```

```
Último año con dato histórico: 2023
Primer año con proyección de OWID: 2024
```

Para esta práctica utilizaremos como punto de separación el **año 2023**. Creamos dos `DataFrames`:

In [104…
```
df_hist = df_mexico[
    df_mexico["Year"] <= 2023
]

df_proy = df_mexico[
    df_mexico["Year"] > 2023
]

print(df_hist.head(3))
print(df_proy.head(3))
```

```
       Entity Code  Year  deaths_estimates  deaths_projected  \
22851  Mexico  MEX  2000          540567.0               NaN
22852  Mexico  MEX  2001          543803.0               NaN
22853  Mexico  MEX  2002          553391.0               NaN

       births_estimates  births_projected     births    deaths
22851         2385977.0               NaN  2385977.0  540567.0
22852         2380261.0               NaN  2380261.0  543803.0
22853         2374140.0               NaN  2374140.0  553391.0
       Entity Code  Year  deaths_estimates  deaths_projected  \
22875  Mexico  MEX  2024               NaN          813823.0
22876  Mexico  MEX  2025               NaN          829063.0
22877  Mexico  MEX  2026               NaN          844431.0

       births_estimates  births_projected     births    deaths
22875               NaN         2023623.0  2023623.0  813823.0
22876               NaN         2003673.0  2003673.0  829063.0
22877               NaN         1989217.0  1989217.0  844431.0
```

In [105…
```
print("Histórico  ->", df_hist.shape, "| años:", df_hist["Year"].min(), "a", df_his
print("Proyectado ->", df_proy.shape, "| años:", df_proy["Year"].min(), "a", df_pro
```

```
Histórico  -> (24, 9) | años: 2000 a 2023
Proyectado -> (77, 9) | años: 2024 a 2100
```

Comprobemos que no haya valores faltantes en las variables que vamos a utilizar para entrenar:

In [106…
```
df_hist[["Year", "births", "deaths"]].isna().sum()
```

Out[106…
```
Year      0
births    0
deaths    0
dtype: int64
```

PARTE 6. Preparar las variables para Machine Learning

11. Definir X e y

Para nuestro primer modelo utilizaremos:

- **X:** año → la **variable predictora**
- **y:** número de nacimientos → la **variable objetivo**

In [107…
```
X = df_hist[["Year"]]
y_nac = df_hist["births"]
```

In [108…
```
X.head()
```

Out[108…

	Year
22851	2000
22852	2001
22853	2002
22854	2003
22855	2004

In [109…
```
y_nac.head()
```

Out[109…
```
22851    2385977.0
22852    2380261.0
22853    2374140.0
22854    2351177.0
22855    2335226.0
Name: births, dtype: float64
```

12. Convertir los datos a NumPy

Obtenemos los valores como arreglos de NumPy:

In [110…
```
X_numpy = X.to_numpy()
y_numpy = y_nac.to_numpy()

print("X_numpy.shape:", X_numpy.shape)
print("y_numpy.shape:", y_numpy.shape)
```

```
X_numpy.shape: (24, 1)
y_numpy.shape: (24,)
```

In [111…
```
type(X_numpy), type(y_numpy)
```

Out[111…
```
(numpy.ndarray, numpy.ndarray)
```

Observación

Scikit-learn espera normalmente que las variables predictoras tengan la forma:

`(número de observaciones, número de variables)`
Es decir, que scikit-learn interpreta X como una tabla de datos.

Por eso `X` tiene **dos dimensiones**, mientras que `y` tiene una sola.

En nuestro ejemplo tenemos:

74 observaciones (registros).

1 variable predictora: año (estas son las caracteristicas o atributos usadas para la predicción).

1 variable objetivo: nacimientos (valor que deseamos predecir).

PARTE 7. Modelo 1 – Regresión lineal

13. Crear el modelo

In [112…
```
mod_lin = LinearRegression()
```

14. Entrenar el modelo

Utilizamos los datos históricos para entrenar el modelo.

In [113…
```
mod_lin.fit(X, y_nac)
```

Out[113…

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

Una regresión lineal aprende una recta `y = a·x + b`. Veamos qué valores aprendió:

In [114…
```
print("Pendiente (a):", mod_lin.coef_[0])
print("Intercepto (b):", mod_lin.intercept_)
```

```
Pendiente (a): -14819.746956521736
Intercepto (b): 32055962.503043473
```

15. Realizar predicciones

Utilizamos el modelo para estimar los nacimientos correspondientes a los **años históricos**.

In [115…
```
preds_lin = mod_lin.predict(X)

preds_lin[:10]
```

Out[115…
```
array([2416468.59      , 2401648.84304348, 2386829.09608696,
       2372009.34913044, 2357189.60217391, 2342369.85521739,
       2327550.10826087, 2312730.36130435, 2297910.61434783,
       2283090.8673913 ])
```

16. Realizar proyecciones hasta 2100

Creamos los años para los que queremos generar **nuestras propias proyecciones**.

In [116…
```
anios_futuros = np.arange(2024, 2101).reshape(-1, 1)

print("Forma:", anios_futuros.shape)
anios_futuros[:10]
```

```
Forma: (77, 1)
```

Out[116…
```
array([[2024],
       [2025],
       [2026],
       [2027],
       [2028],
       [2029],
       [2030],
       [2031],
       [2032],
       [2033]])
```

> **Detalle técnico:** entrenamos el modelo con `X`, que es un `DataFrame` con una columna llamada `Year`. Scikit-learn recuerda ese nombre, así que para predecir también debemos entregarle un `DataFrame` con una columna `Year`. Si le pasáramos un arreglo de NumPy "a secas" mostraría una advertencia (*UserWarning: X does not have valid feature names*). Por eso convertimos los años futuros a `DataFrame`.

In [117…
```
X_futuro = pd.DataFrame(anios_futuros, columns=["Year"])

X_futuro.head()
```

Out[117…

	Year
0	2024
1	2025
2	2026
3	2027
4	2028

In [118…
```
preds_lin_futuras = mod_lin.predict(X_futuro)
```

In [119…
```
print(preds_lin_futuras[:10])
```

```
[2060794.66304348 2045974.91608696 2031155.16913044 2016335.42217391
 2001515.67521739 1986695.92826087 1971876.18130435 1957056.43434783
 1942236.6873913  1927416.94043478]
```

PARTE 8. Visualizar la regresión lineal

17. Comparar datos históricos y predicciones

In [120…
```
plt.figure(figsize=(12, 6))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

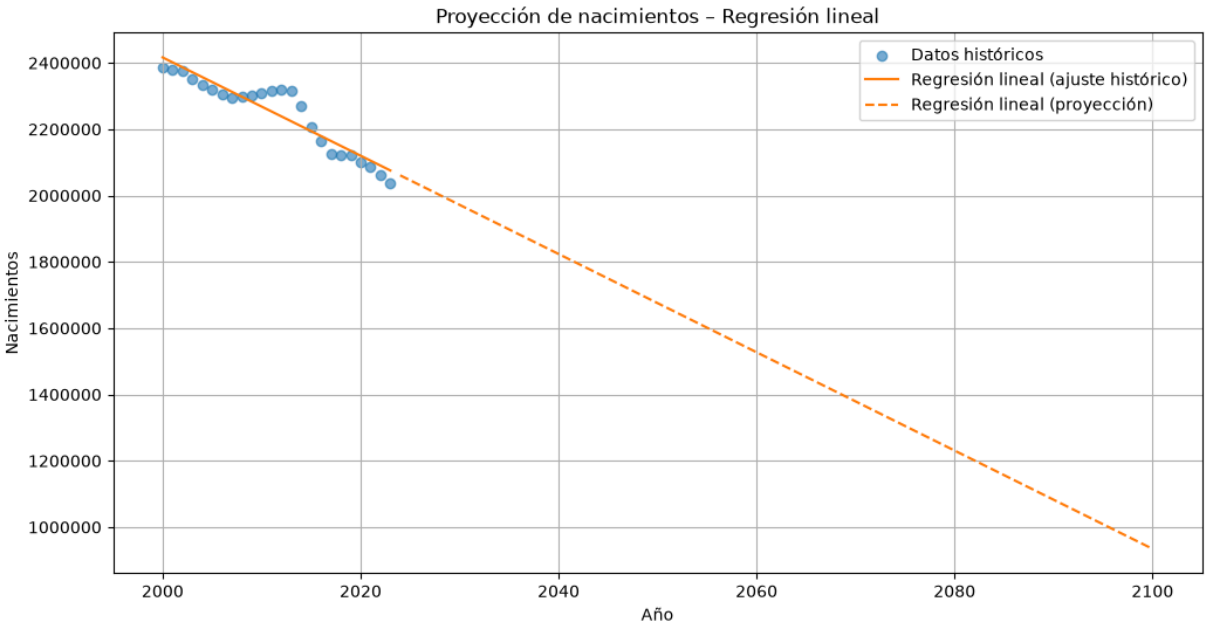
# Ajuste del modelo sobre los años históricos
plt.plot(
    df_hist["Year"],
    preds_lin,
    color="tab:orange",
```

```
    label="Regresión lineal (ajuste histórico)"
)

# Proyección hacia el futuro
plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    linestyle="--",
    label="Regresión lineal (proyección)"
)

plt.title("Proyección de nacimientos - Regresión lineal")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Analiza cómo continúa la tendencia histórica cuando utilizamos una **función lineal**. ¿Qué pasa con los valores hacia el año 2100? Pues la regresion lineal solo dibuja una linea recta hacia abajo ignorando que la caida no es tan recta. Hacia el 2100 predice que los nacimientos van a seguir bajando hasta llegar a cero o a negativos, lo cual no tiene sentido en la vida real.

PARTE 9. Modelo 2 – Regresión polinomial

18. Crear variables polinomiales

La regresión lineal solamente puede representar una relación **lineal** entre las variables. Ahora construiremos variables polinomiales para permitir **curvas**. Utilizaremos un polinomio de **grado 2**:

$$y = a \cdot x^2 + b \cdot x + c$$

In [121…
```
poly = PolynomialFeatures(
    degree=2,
    include_bias=False
)
```

Transformamos `X`:

In [122…
```
X_poly = poly.fit_transform(X)

X_poly[:5]
```

Out[122…
```
array([[2.000000e+03, 4.000000e+06],
       [2.001000e+03, 4.004001e+06],
       [2.002000e+03, 4.008004e+06],
       [2.003000e+03, 4.012009e+06],
       [2.004000e+03, 4.016016e+06]])
```

Observa que ahora cada año tiene **dos columnas**: el año (`x`) y el año al cuadrado (`x²`).

19. Crear y entrenar el modelo polinomial

Seguimos utilizando `LinearRegression`; lo único que cambió son las **variables de entrada**.

In [123…
```
mod_poli = LinearRegression()

mod_poli.fit(
    X_poly,
    y_nac
)
```

Out[123…

▾ LinearRegression ⓘ ?

▸ Parameters

▸ Fitted attributes

20. Generar predicciones históricas

In [124…
```
preds_poli = mod_poli.predict(
    X_poly
)
```

21. Generar proyecciones hasta 2100

Transformamos los años futuros utilizando **el mismo objeto** `poly` con el que transformamos los datos de entrenamiento.

In [125…
```
anios_futuros_poly = poly.transform(
    X_futuro
)

preds_poli_futuras = mod_poli.predict(
    anios_futuros_poly
)
```

PARTE 10. Visualizar la regresión polinomial

22. Graficar las predicciones

In [126…
```
plt.figure(figsize=(12, 6))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

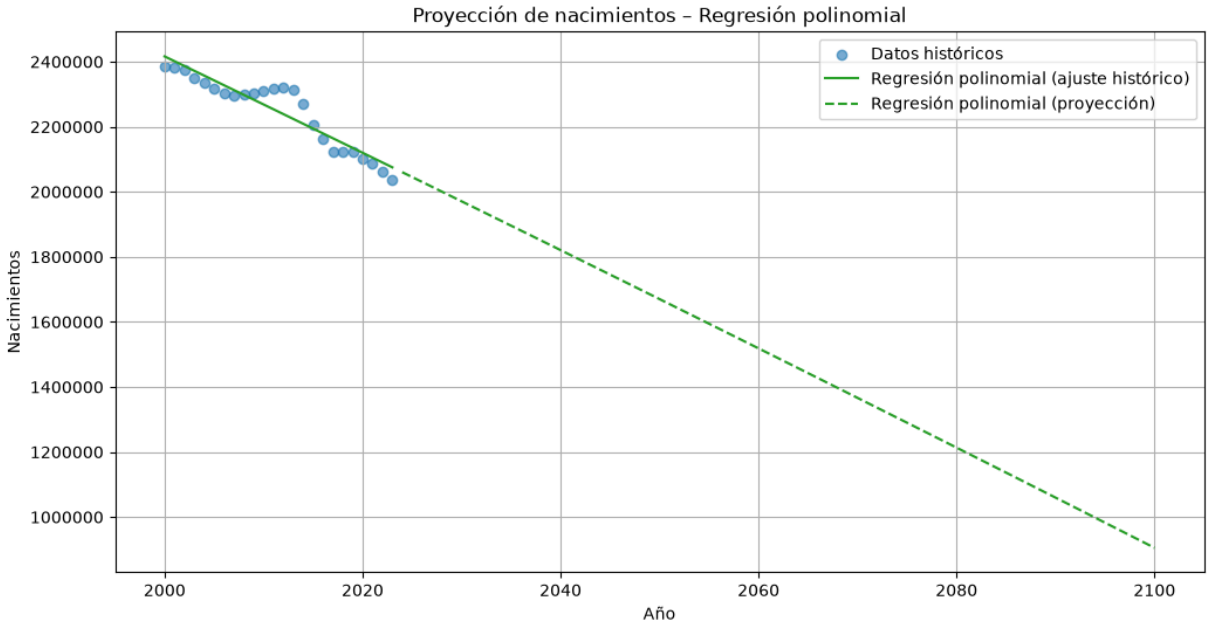
plt.plot(
    df_hist["Year"],
    preds_poli,
    color="tab:green",
    label="Regresión polinomial (ajuste histórico)"
)

plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    linestyle="--",
    label="Regresión polinomial (proyección)"
)

plt.title("Proyección de nacimientos – Regresión polinomial")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
```

```
plt.legend()
plt.grid()
plt.show()
```



Observación

Compara el comportamiento de esta gráfica con el modelo de regresión lineal. ¿Se ajusta mejor a los datos históricos? Si se ve mucho mejor porque la curvita se va adaptando a las variaciones de los años pasados. A diferencia de la recta, copia un poco mejor la subida y la caida.

¿Qué ocurre con la proyección hacia el año 2100? ¿Los valores que predice tienen sentido para un número de nacimientos? Aqui ya se va todo a la basura jaja. La proyeccion se va super rapido para abajo a numeros negativos desde mucho antes del 2100. Es obvio que no tiene sentido tener -1 millón de nacimientos, la regresion polinomial exagera mucho la caida y se vuelve inutil.

PARTE 11. Modelo 3 – Random Forest

23. Crear el modelo

Un **Random Forest** está formado por muchos árboles de decisión. Utilizaremos `RandomForestRegressor`.

In [127…

```
mod_rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
```

24. Entrenar el modelo

In [128…
```
mod_rf.fit(
    X,
    y_nac
)
```

Out[128…
RandomForestRegressor

▶ Parameters

▶ Fitted attributes

25. Generar predicciones históricas

In [129…
```
preds_rf = mod_rf.predict(X)
```

26. Generar proyecciones futuras

In [130…
```
preds_rf_futuras = mod_rf.predict(
    X_futuro
)
```

PARTE 12. Visualizar Random Forest

27. Graficar las predicciones

In [131…
```
plt.figure(figsize=(12, 6))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.6
)

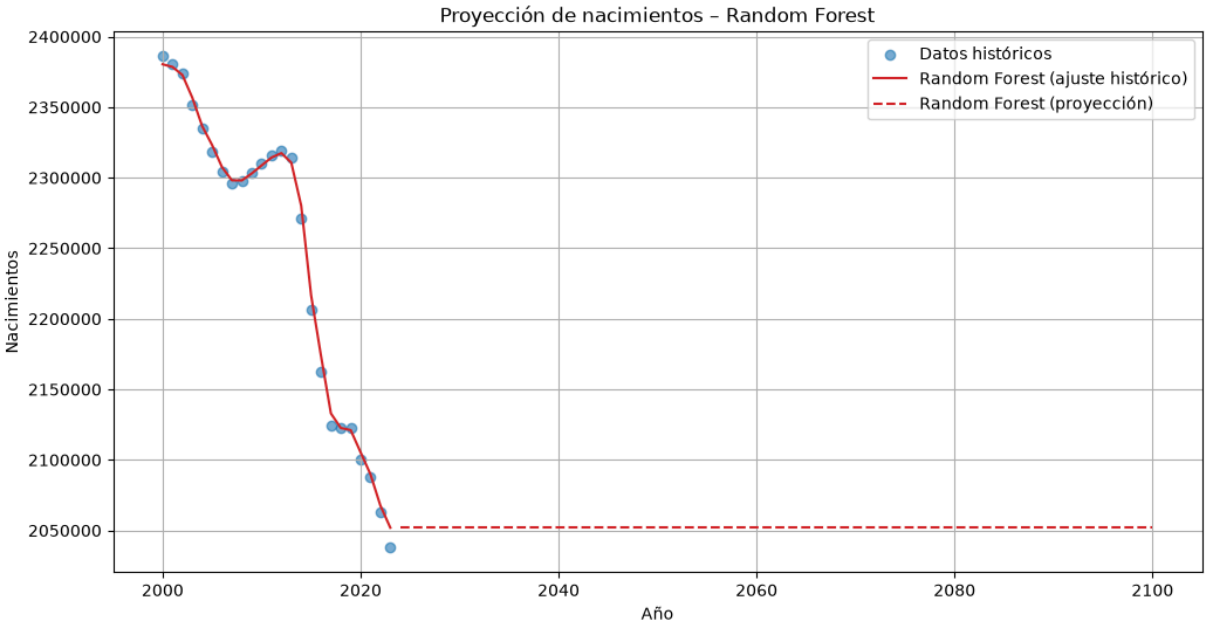
plt.plot(
    df_hist["Year"],
    preds_rf,
    color="tab:red",
    label="Random Forest (ajuste histórico)"
)

plt.plot(
```

```
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    linestyle="--",
    label="Random Forest (proyección)"
)

plt.title("Proyección de nacimientos – Random Forest")
plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



Observación

Fíjate en dos cosas: (1) qué tan bien sigue el modelo los datos históricos y (2) qué sucede con la línea después de 2023. ¿Continúa la tendencia? ¿Por qué crees que sucede esto? Investiga.

1. El modelo sigue los datos historicos como si fuera un calcado. Pasa por cada puntito sin fallar (creo que a esto le dicen overfitting o algo asi).
2. Despues del 2023 la linea se queda completamente plana. No sigue ni para arriba ni para abajo. Me puse a buscar y al parecer los random forest no pueden predecir mas alla del rango que ya vieron, asi que solo repiten el ultimo valor que aprendieron.

PARTE 13. Comparar los tres modelos

28. Graficar los tres modelos

Construimos una gráfica que permita comparar directamente los resultados.

> Si la regresión polinomial genera valores muy extremos y no permite ver bien las demás curvas, puedes limitar el eje vertical agregando, por ejemplo, `plt.ylim(0, 3_500_000)` antes de `plt.show()`.

In [132…

```
plt.figure(figsize=(14, 7))

plt.scatter(
    df_hist["Year"],
    df_hist["births"],
    label="Datos históricos",
    alpha=0.5
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    label="Regresión lineal"
)

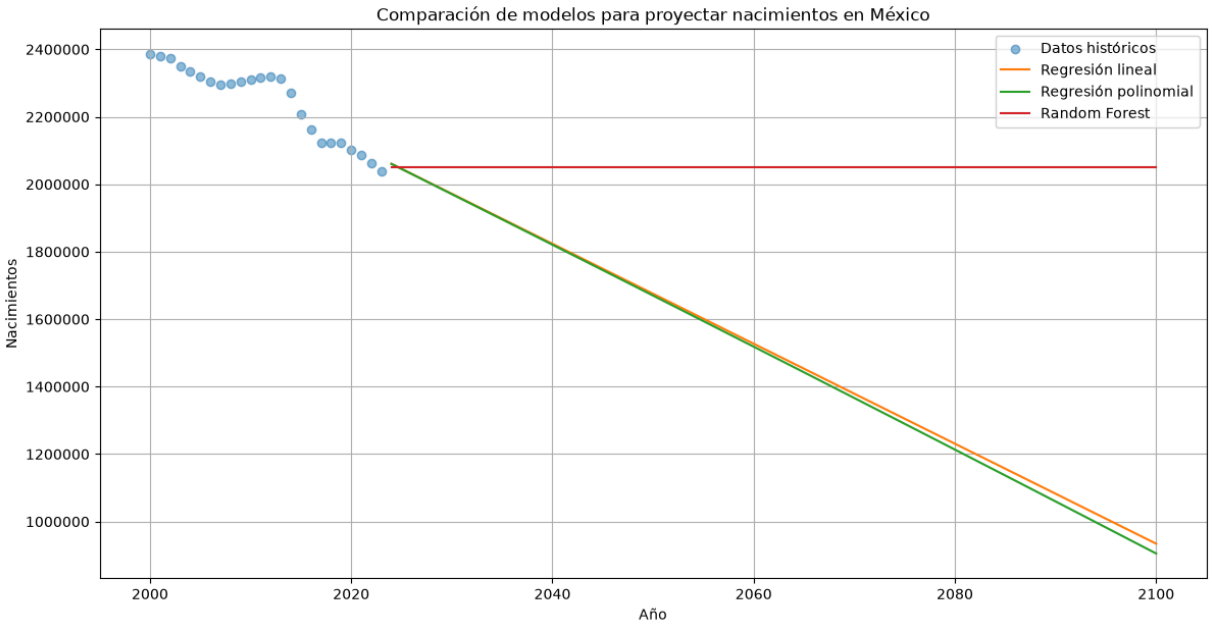
plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Comparación de modelos para proyectar nacimientos en México"
)

plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



PARTE 14. Repetir el proceso para defunciones

Hasta este momento construimos modelos para predecir **nacimientos**. Ahora repetiremos el mismo proceso para **defunciones**.

29. Preparar la variable objetivo

La variable predictora sigue siendo `X` (el año). Solo cambia la variable objetivo:

In [133…
```
y_def = df_hist["deaths"]
```

30. Regresión lineal para defunciones

In [134…
```
mod_lin_deaths = LinearRegression()

mod_lin_deaths.fit(
    X,
    y_def
)

preds_lin_deaths = mod_lin_deaths.predict(
    X_futuro
)
```

31. Regresión polinomial para defunciones

Utilizamos nuevamente las variables construidas con `PolynomialFeatures` (`X_poly` y `anios_futuros_poly`).

In [135…
```
mod_poli_deaths = LinearRegression()

mod_poli_deaths.fit(
    X_poly,
    y_def
)

preds_poli_deaths = mod_poli_deaths.predict(
    anios_futuros_poly
)
```

32. Random Forest para defunciones

In [136…
```
mod_rf_deaths = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

mod_rf_deaths.fit(
    X,
    y_def
)

preds_rf_deaths = mod_rf_deaths.predict(
    X_futuro
)
```

PARTE 15. Comparar las predicciones de defunciones

33. Visualizar los tres modelos

In [137…
```
plt.figure(figsize=(14, 7))

plt.scatter(
    df_hist["Year"],
    df_hist["deaths"],
    label="Datos históricos",
    alpha=0.5
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_deaths,
    color="tab:orange",
    label="Regresión lineal"
```

```
)

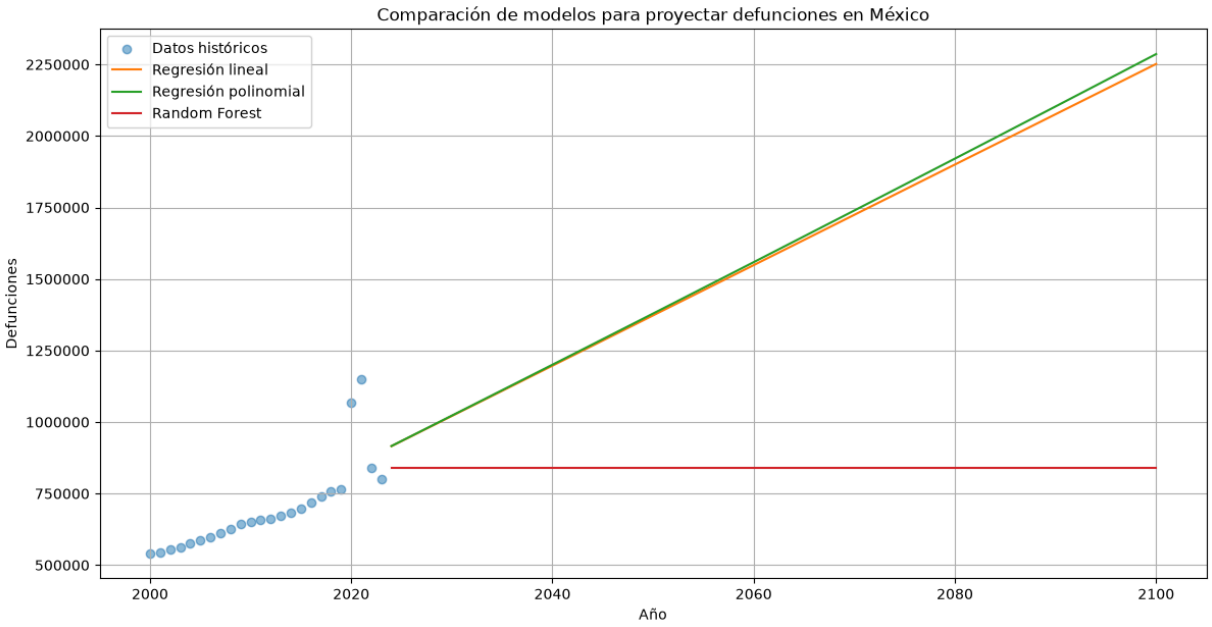
plt.plot(
    anios_futuros.flatten(),
    preds_poli_deaths,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_deaths,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Comparación de modelos para proyectar defunciones en México"
)

plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



PARTE 16. Comparar nuestros modelos con las proyecciones de OWID

Finalmente, agregamos a las gráficas las **proyecciones proporcionadas por OWID** (las de la ONU, escenario medio). Estas proyecciones no se obtienen ajustando una recta a la serie

histórica: se construyen con modelos demográficos que consideran fecundidad, mortalidad y estructura por edad de la población.

34. Nacimientos: ML vs OWID

In [138…

```
plt.figure(figsize=(14, 7))

plt.plot(
    df_proy["Year"],
    df_proy["births"],
    color="black",
    linewidth=2.5,
    label="Proyección OWID"
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_futuras,
    color="tab:orange",
    label="Regresión lineal"
)

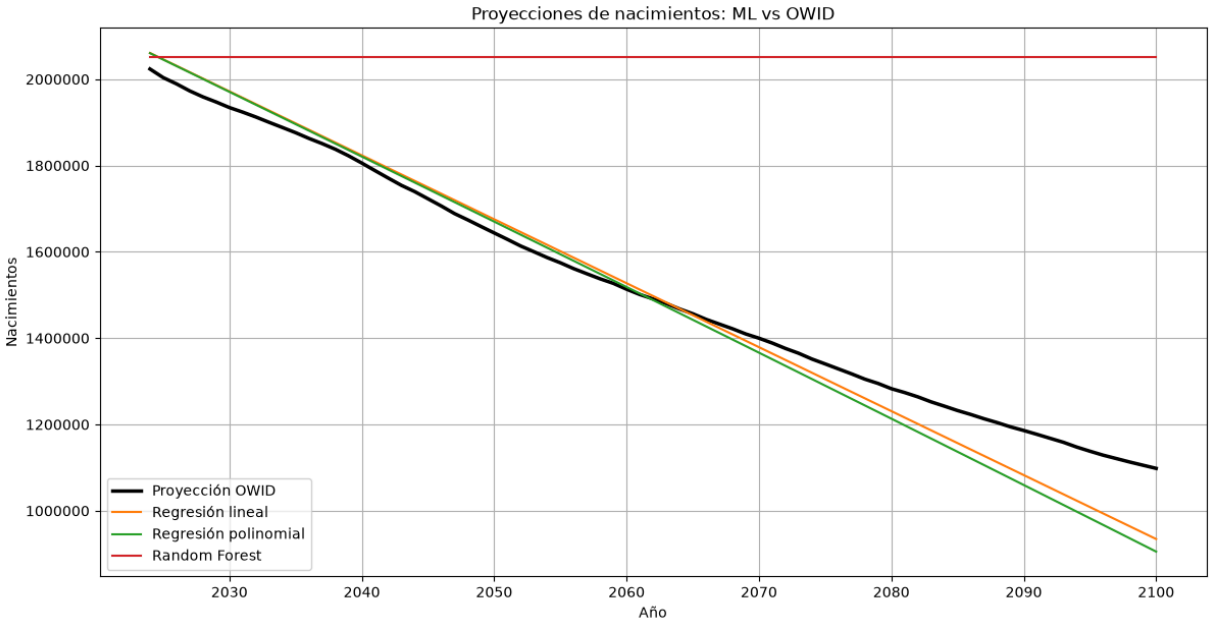
plt.plot(
    anios_futuros.flatten(),
    preds_poli_futuras,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_futuras,
    color="tab:red",
    label="Random Forest"
)

plt.title(
    "Proyecciones de nacimientos: ML vs OWID"
)

plt.xlabel("Año")
plt.ylabel("Nacimientos")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



35. Defunciones: ML vs OWID

Repetimos el ejercicio para defunciones.

In [139…

```
plt.figure(figsize=(14, 7))

plt.plot(
    df_proy["Year"],
    df_proy["deaths"],
    color="black",
    linewidth=2.5,
    label="Proyección OWID"
)

plt.plot(
    anios_futuros.flatten(),
    preds_lin_deaths,
    color="tab:orange",
    label="Regresión lineal"
)

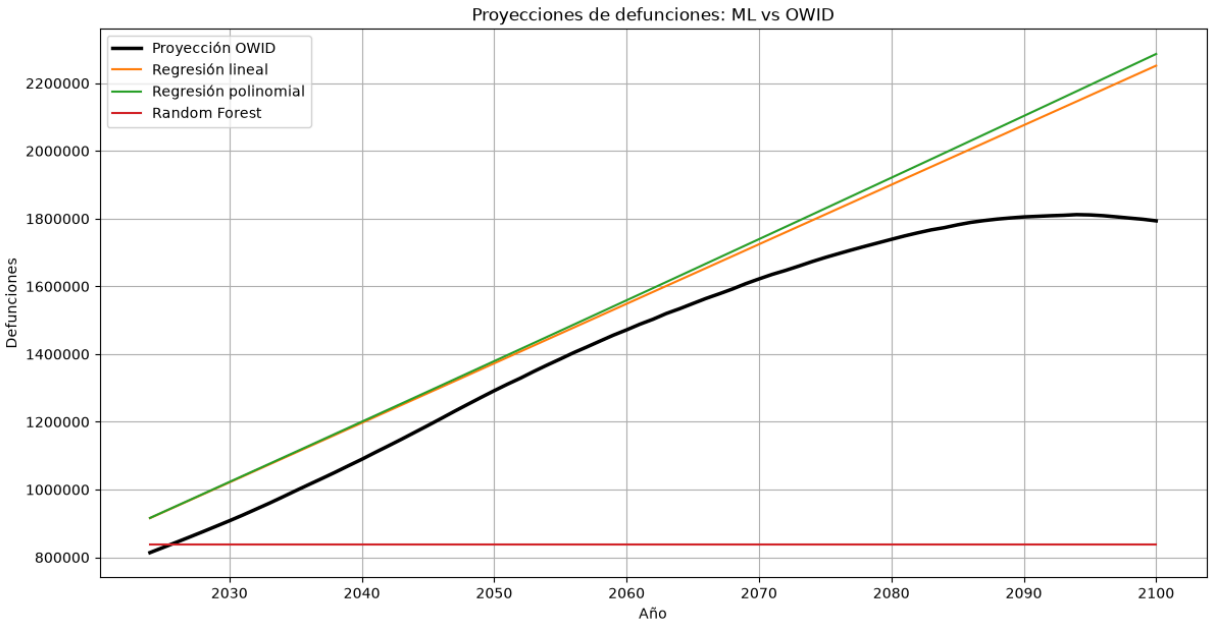
plt.plot(
    anios_futuros.flatten(),
    preds_poli_deaths,
    color="tab:green",
    label="Regresión polinomial"
)

plt.plot(
    anios_futuros.flatten(),
    preds_rf_deaths,
    color="tab:red",
    label="Random Forest"
)
```

```
plt.title(
    "Proyecciones de defunciones: ML vs OWID"
)

plt.xlabel("Año")
plt.ylabel("Defunciones")

plt.ticklabel_format(style="plain", axis="y", useOffset=False)
plt.legend()
plt.grid()
plt.show()
```



36. Resumen numérico (opcional)

A veces es útil ver los números y no solo la gráfica. Esta tabla muestra lo que predice cada modelo para algunos años seleccionados:

In [140…

```
anios_tabla = [2030, 2050, 2075, 2100]
indice = anios_futuros.flatten()


def construir_tabla(owid, lineal, polinomial, rf):
    tabla = pd.DataFrame(
        {
            "OWID": owid.set_index("Year").reindex(indice).to_numpy().ravel(),
            "Regresión lineal": lineal,
            "Regresión polinomial": polinomial,
            "Random Forest": rf,
        },
        index=indice
    )
    tabla.index.name = "Año"
    return tabla.loc[anios_tabla]
```

```
print("NACIMIENTOS")
display(
    construir_tabla(
        df_proy[["Year", "births"]],
        preds_lin_futuras,
        preds_poli_futuras,
        preds_rf_futuras
    ).style.format("{:,.0f}")
)

print("DEFUNCIONES")
display(
    construir_tabla(
        df_proy[["Year", "deaths"]],
        preds_lin_deaths,
        preds_poli_deaths,
        preds_rf_deaths
    ).style.format("{:,.0f}")
)
```

NACIMIENTOS

Año	OWID	Regresión lineal	Regresión polinomial	Random Forest
2030	1,934,285	1,971,876	1,970,689	2,051,955
2050	1,643,767	1,675,481	1,669,982	2,051,955
2075	1,340,246	1,304,988	1,289,953	2,051,955
2100	1,098,210	934,494	905,317	2,051,955

DEFUNCIONES

Año	OWID	Regresión lineal	Regresión polinomial	Random Forest
2030	907,843	1,021,129	1,022,528	838,793
2050	1,291,889	1,372,705	1,379,209	838,793
2075	1,685,181	1,812,175	1,829,978	838,793
2100	1,793,462	2,251,646	2,286,211	838,793

In [147…

```
import csv

df_base = pd.read_csv("C:\\Users\\Santiago\\Desktop\\Escuela\\5to Semestre ISSC\\Bi

    # 2. Definir las URLs de los nuevos datos de Our World in Data
    # Usamos el parámetro csvType=full para traer el histórico completo
url_gdp = "https://ourworldindata.org/grapher/gdp-worldbank.csv?v=1&csvType=full&us
url_health = "https://ourworldindata.org/grapher/public-health-expenditure-share-gd
```

```
opciones_owid = {"User-Agent": "Our World In Data data fetch/1.0"}

    # Descargar los DataFrames
df_gdp = pd.read_csv(url_gdp, storage_options=opciones_owid)
df_health = pd.read_csv(url_health, storage_options=opciones_owid)

    # Estandarizar nombres de columnas a minúsculas por si acaso
df_gdp.columns = df_gdp.columns.str.lower()
df_health.columns = df_health.columns.str.lower()

    # 3. Detectar dinámicamente el nombre de la columna de datos
    # OWID a veces cambia los sufijos de sus columnas, así que filtramos 'entity',
col_gdp = [c for c in df_gdp.columns if c not in ["entity", "code", "year"]][0]
col_health = [c for c in df_health.columns if c not in ["entity", "code", "year"]][

    # Extraemos solo lo necesario y renombramos al formato solicitado
df_gdp_limpio = df_gdp[['entity', 'year', col_gdp]].rename(columns={col_gdp: 'gdp'}
df_health_limpio = df_health[['entity', 'year', col_health]].rename(columns={col_he
'percentage_health'})

    # 4. Unir (Merge) los datasets
    # Unimos GDP al dataset base
df_final = pd.merge(df_base, df_gdp_limpio, on=['entity', 'year'], how='inner')
    # Unimos Health al resultado anterior
df_final = pd.merge(df_final, df_health_limpio, on=['entity', 'year'], how='inner')

    # 5. Filtrar solo a China y México
df_final = df_final[df_final['entity'].isin(['China', 'Mexico'])]

    # 6. Seleccionar estrictamente las 8 columnas requeridas por el ejercicio
columnas_requeridas = [
        'entity', 'year', 'population', 'births', 'deaths',
        'child_deaths_under_5', 'percentage_health', 'gdp'
    ]
df_final = df_final[columnas_requeridas]

    # 7. Guardar el nuevo dataset resultante
df_final.to_csv("../data/processed/demografia_china_mexico_completo.csv", index=Fal
print("Dimensiones del dataset final:", df_final.shape)
display(df_final.head())
```

```
Dimensiones del dataset final: (46, 8)
```

	entity	year	population	births	deaths	child_deaths_under_5	percentage_he
226	China	2000	1.269581e+09	17551192.0	7780055.0	625893	0.
227	China	2001	1.278725e+09	16586483.0	7830809.0	561863	0.
228	China	2002	1.286867e+09	16382375.0	7916788.0	512491	1.
229	China	2003	1.294517e+09	16221728.0	8011829.0	467493	1.
230	China	2004	1.302100e+09	16461427.0	8121151.0	433014	1.